

ВВЕДЕНИЕ В МАРШРУТ ПРОЕКТИРОВАНИЯ И УПРАЖНЕНИЯ С КОМБИНАЦИОННОЙ ЛОГИКОЙ



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ

ПРИ ПАРТНЕРСТВЕ



Александр Силантьев
Руководитель лаборатории
НИЛ ЭСК Университета МИЭТ



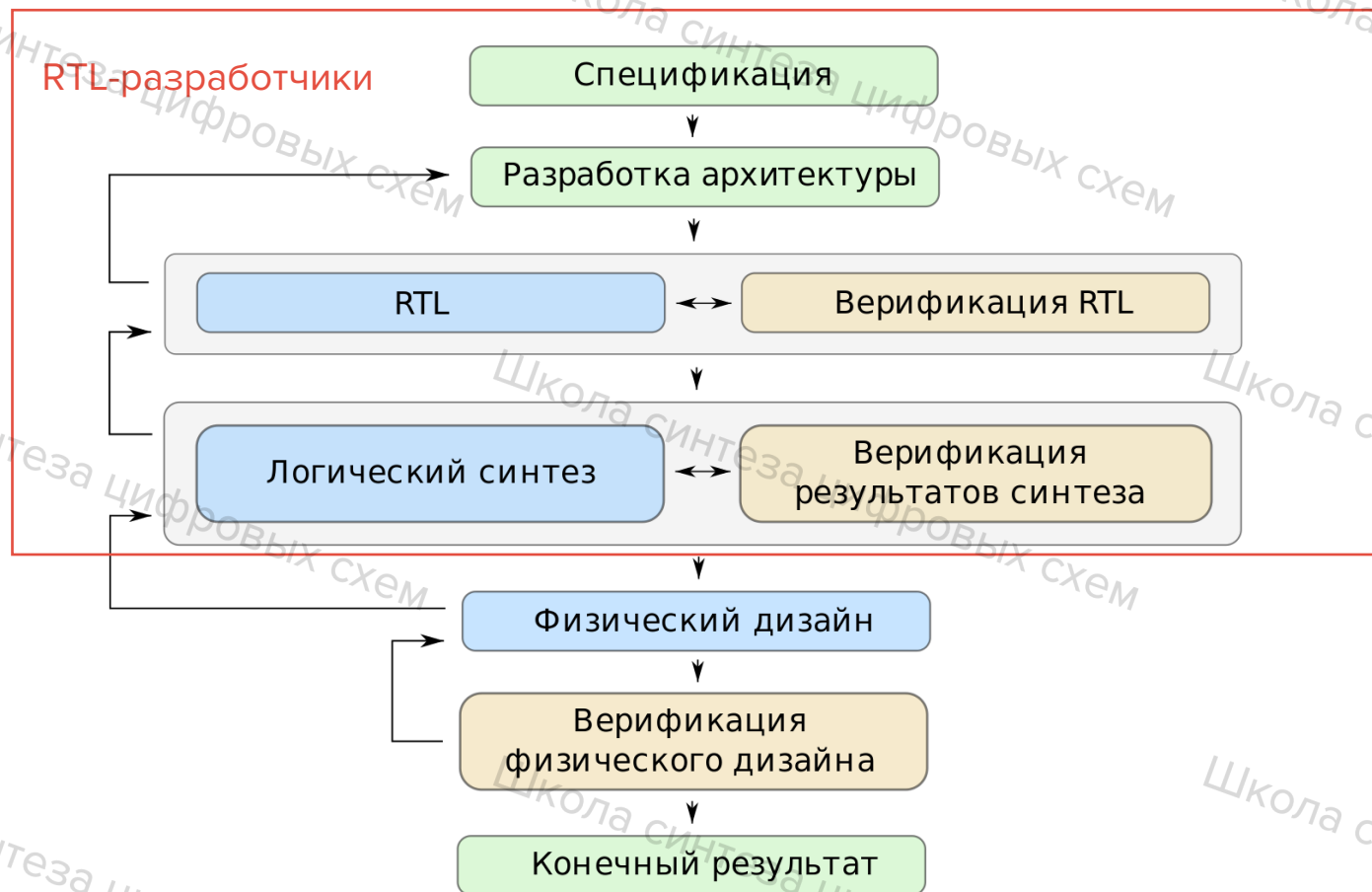
Александр Силантьев

Руководитель лаборатории НИЛ ЭСК Университета МИЭТ

- Окончил МИЭТ в 2014 году
- 14-ти летний опыт инженерной деятельности в области проектирования IP-ядер и СнК.
- Старший преподаватель института МПСУ МИЭТ.
- Организатор первого в России инженерного хакатона по микроэлектронике и системам на кристалле.
- С 2014 года организатор семинаров, школ и олимпиад по популяризации электроники среди студентов и школьников

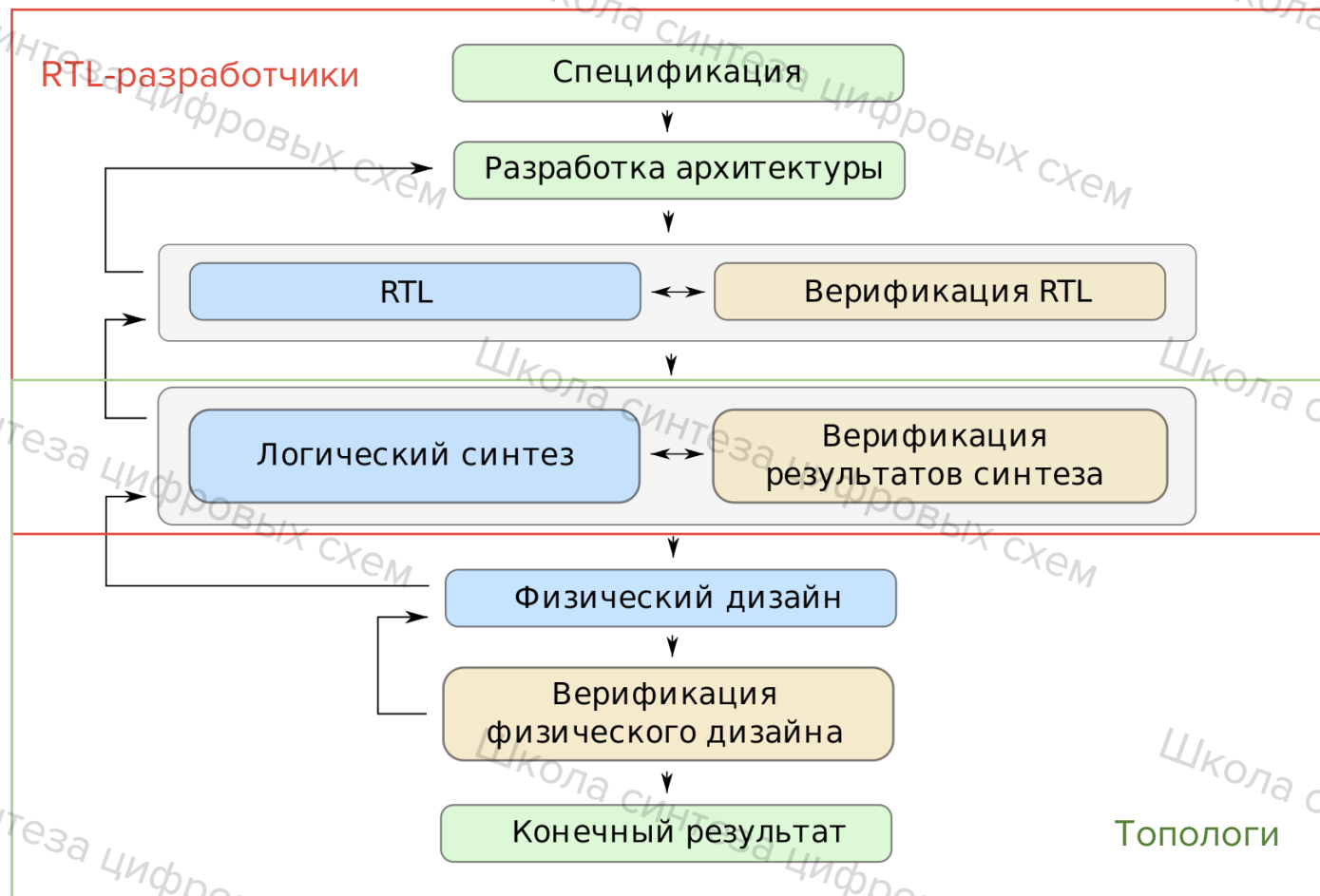


МАРШРУТ ПРОЕКТИРОВАНИЯ ЦИФРОВЫХ УСТРОЙСТВ

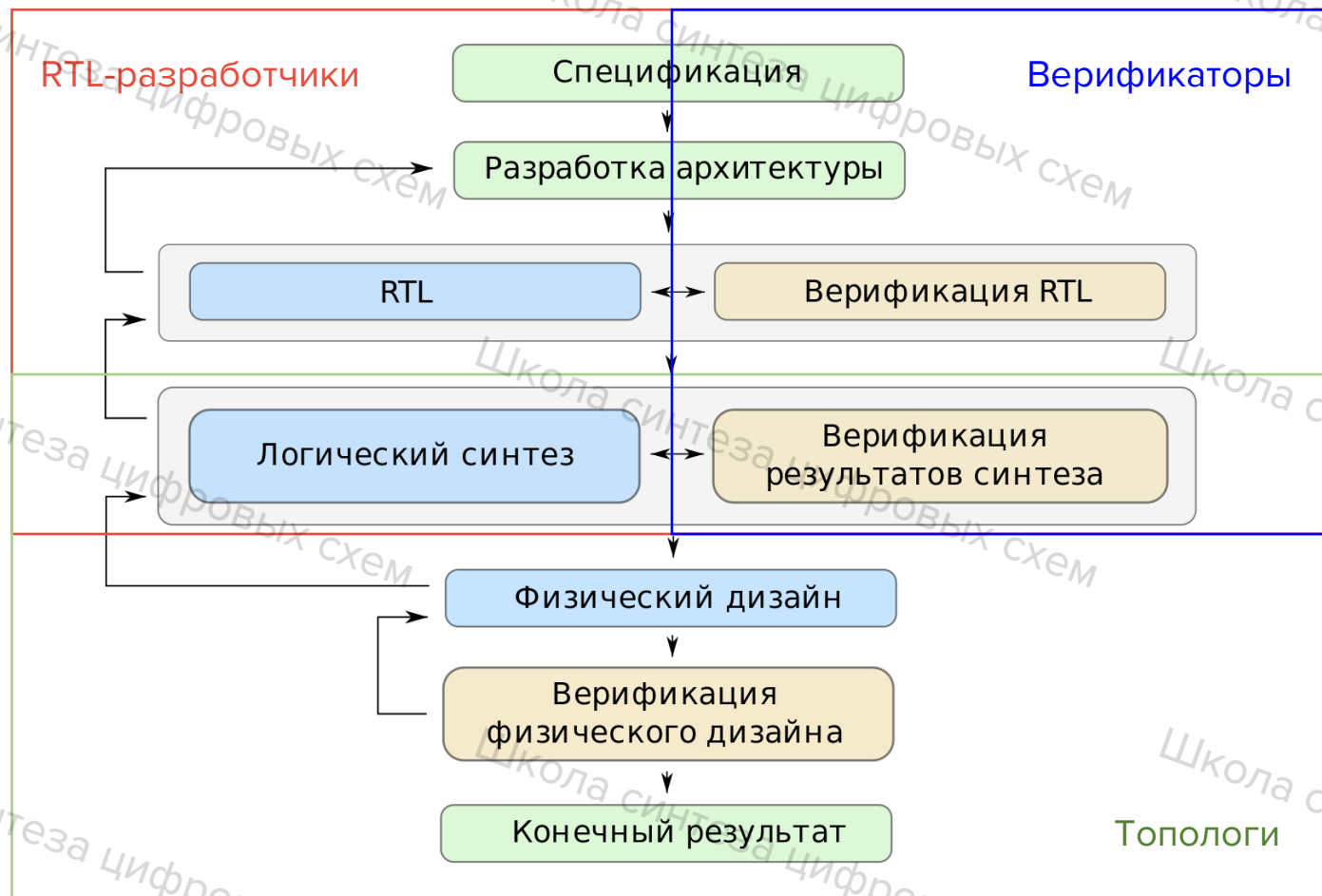




МАРШРУТ ПРОЕКТИРОВАНИЯ ЦИФРОВЫХ УСТРОЙСТВ

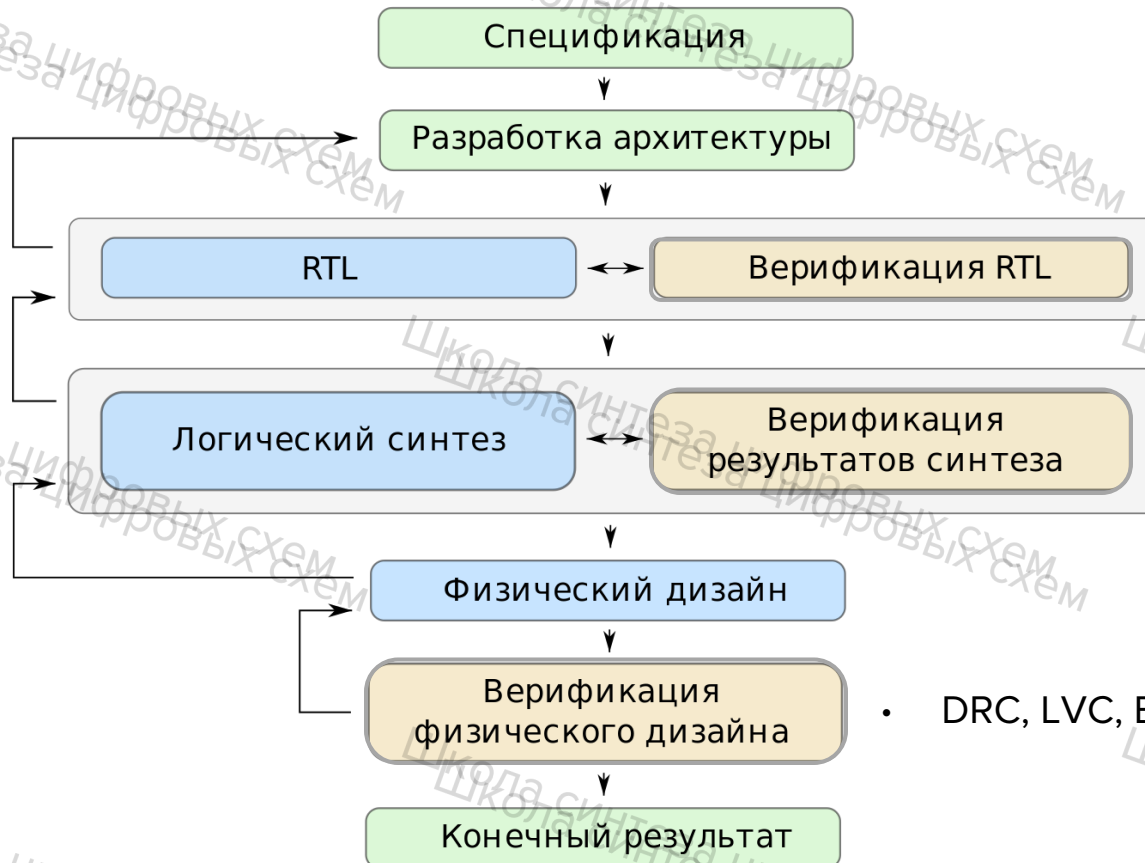


МАРШРУТ ПРОЕКТИРОВАНИЯ ЦИФРОВЫХ УСТРОЙСТВ





МАРШРУТ ПРОЕКТИРОВАНИЯ ЦИФРОВЫХ УСТРОЙСТВ



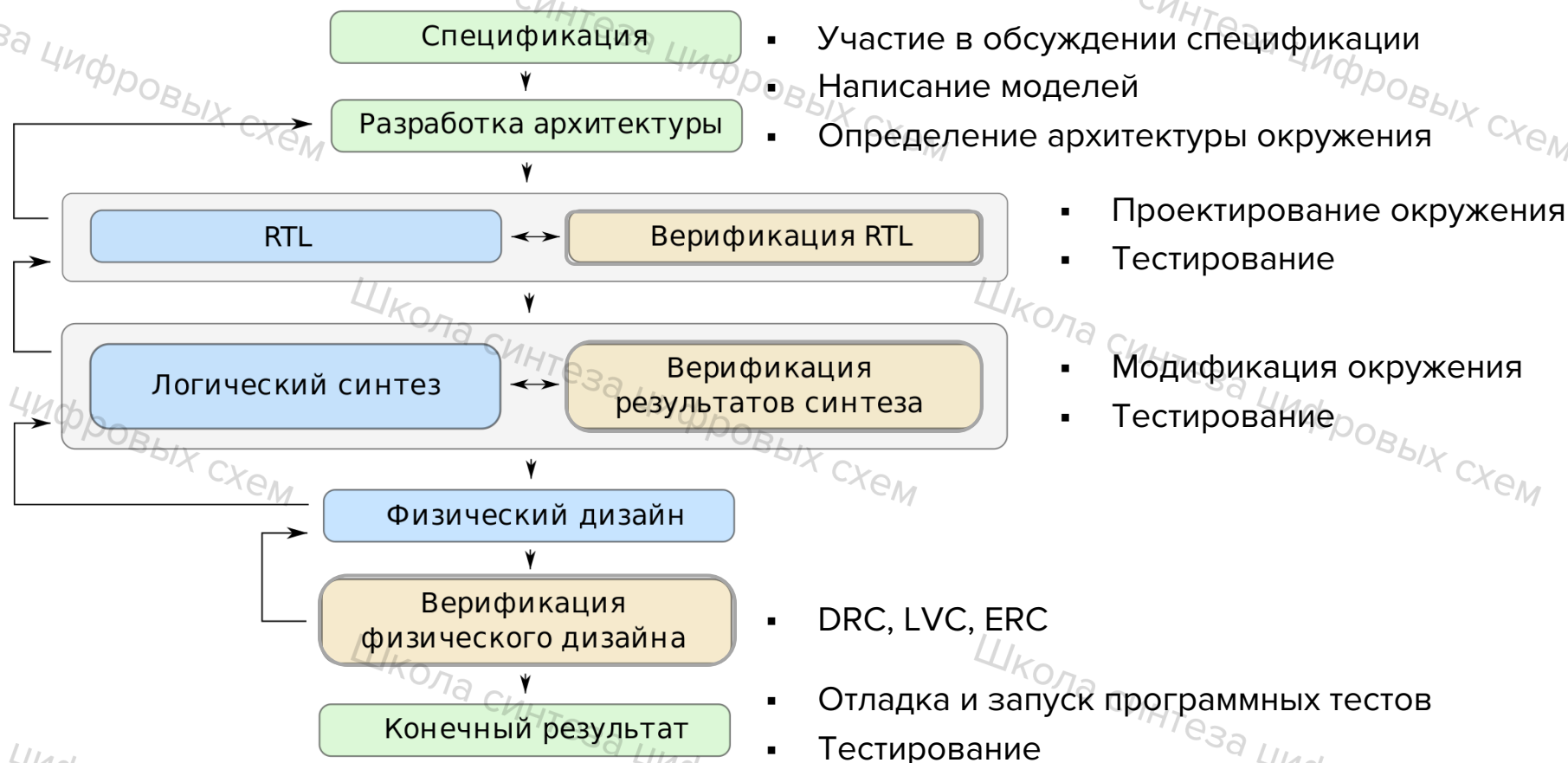
- Проектирование окружения
- Тестирование

- Модификация окружения
- Тестирование

- DRC, LVC, ERC

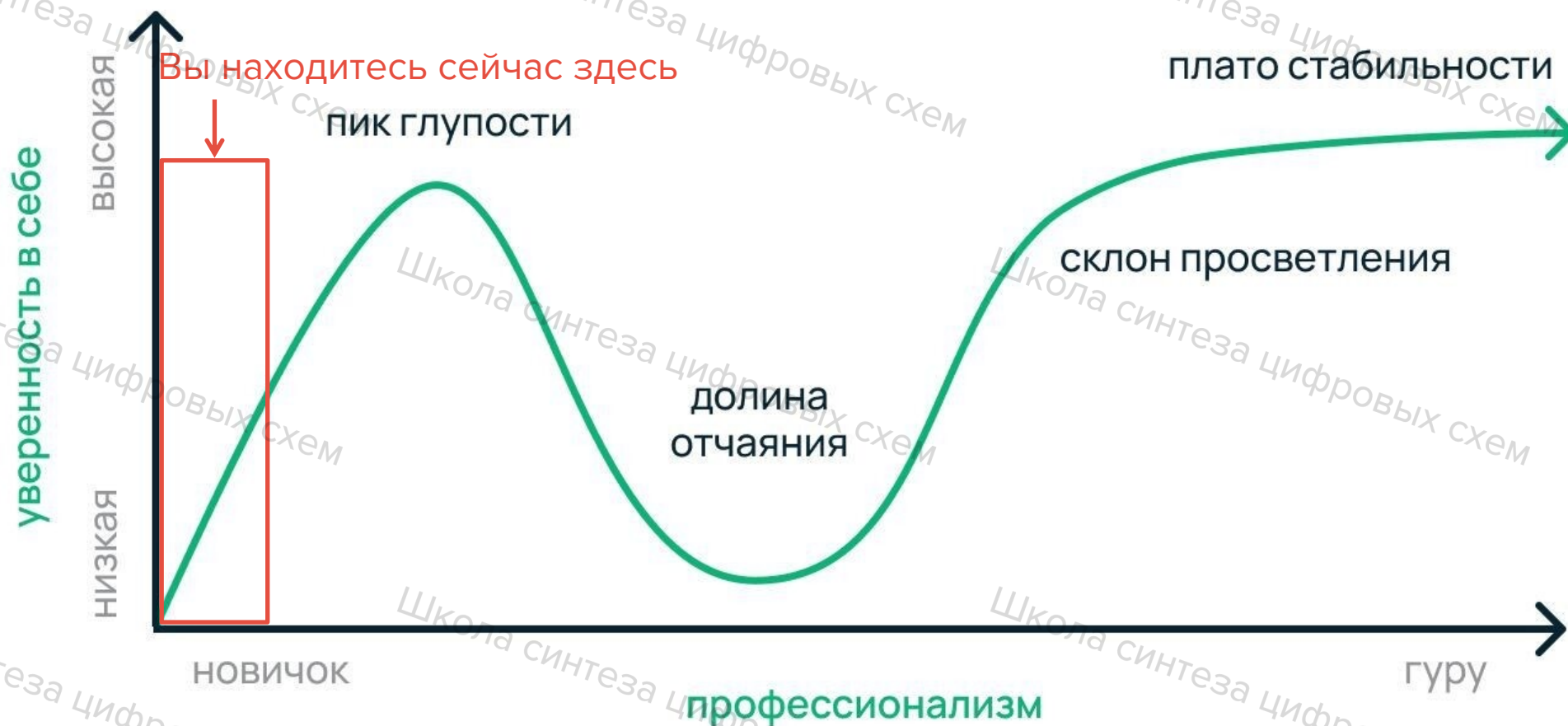


МАРШРУТ ПРОЕКТИРОВАНИЯ ЦИФРОВЫХ УСТРОЙСТВ





ЭФФЕКТ ДАНИНГА-КРЮГЕРА. ЗАНИМАТЕЛЬНЫЕ «АМЕРИКАНСКИЕ ГОРКИ»





ЭФФЕКТ ДАНИНГА-КРЮГЕРА. ЗАНИМАТЕЛЬНЫЕ «АМЕРИКАНСКИЕ ГОРКИ»





ЭФФЕКТ ДАНИНГА-КРЮГЕРА. ЗАНИМАТЕЛЬНЫЕ «АМЕРИКАНСКИЕ ГОРКИ»



ПОЧЕМУ ШКОЛА СИНТЕЗА УСТРОЕНА ИМЕННО ТАК



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ

Школа Синтеза даёт начальные знания и кругозор —

вы познакомитесь с верификацией, микроархитектурой процессоров, проектированием микросхем ASIC.

Школа не замена университетским курсам,
а еще одна возможность для роста.

Пересматривайте лекции, если что-то не поняли сразу

(это нормально, объём информации большой) — у вас будут записи каждого занятия

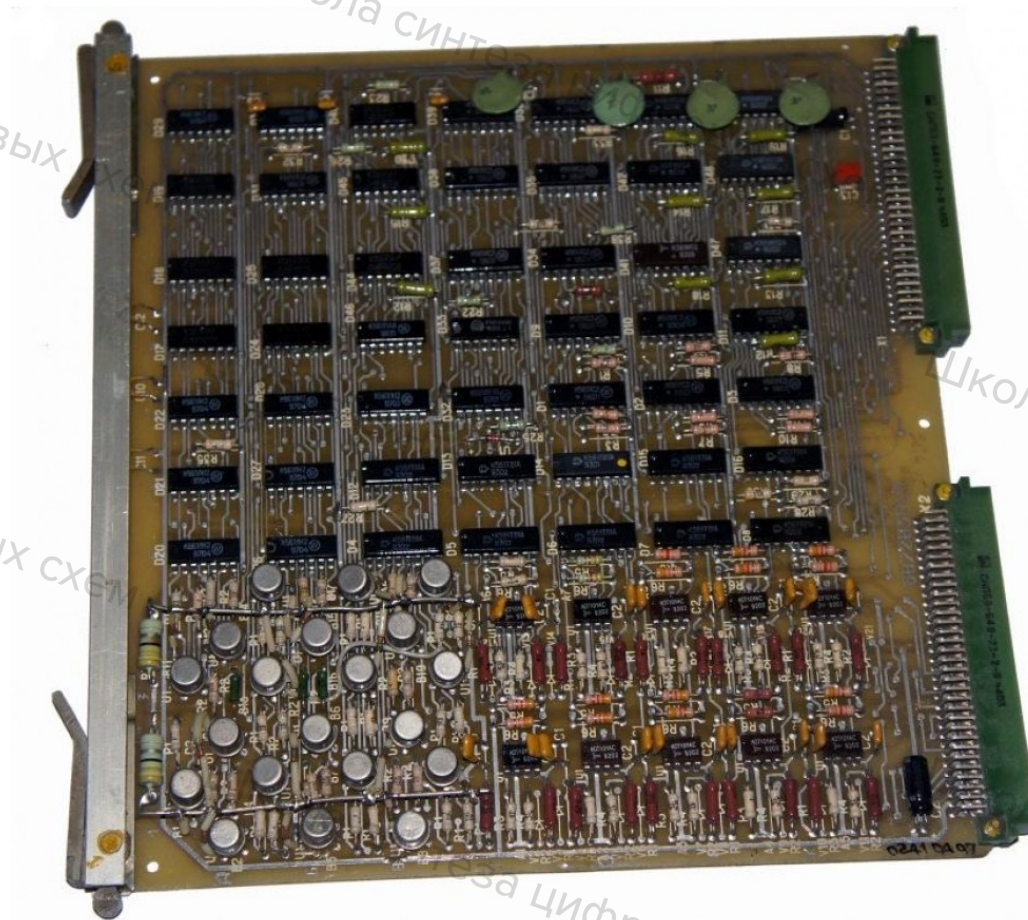
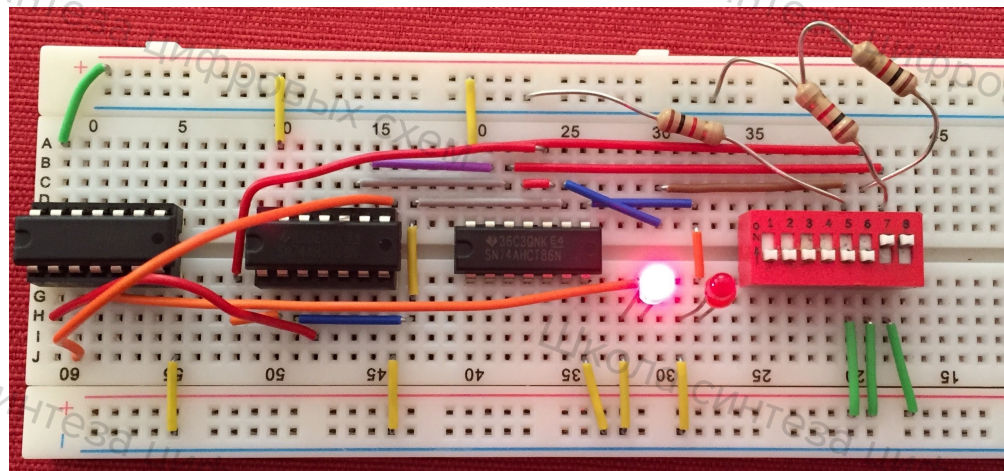
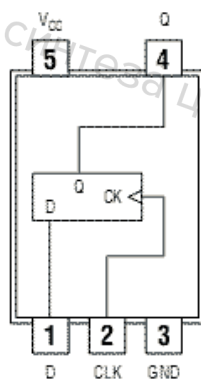
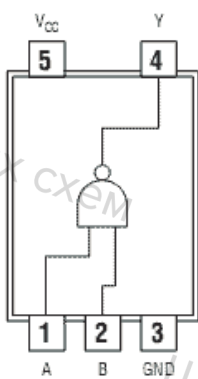
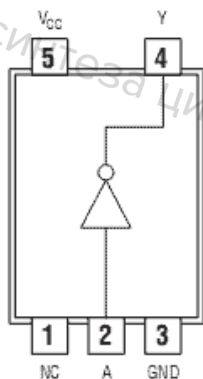
Чтобы действительно освоить темы Школы Синтеза, советуем заниматься и самостоятельно вне занятий Школы —

чем больше вы пишете код и проектируете сами, тем быстрее прокачаетесь и вырастаете как специалист

Если вам что-то непонятно, задавайте вопрос на лекции или в телеграм-чате.

Если вопрос кажется глупым — тем более спрашивайте

МИКРОСХЕМЫ МАЛОЙ СТЕПЕНИ ИНТЕГРАЦИИ



SCHEMA INTEL 4004

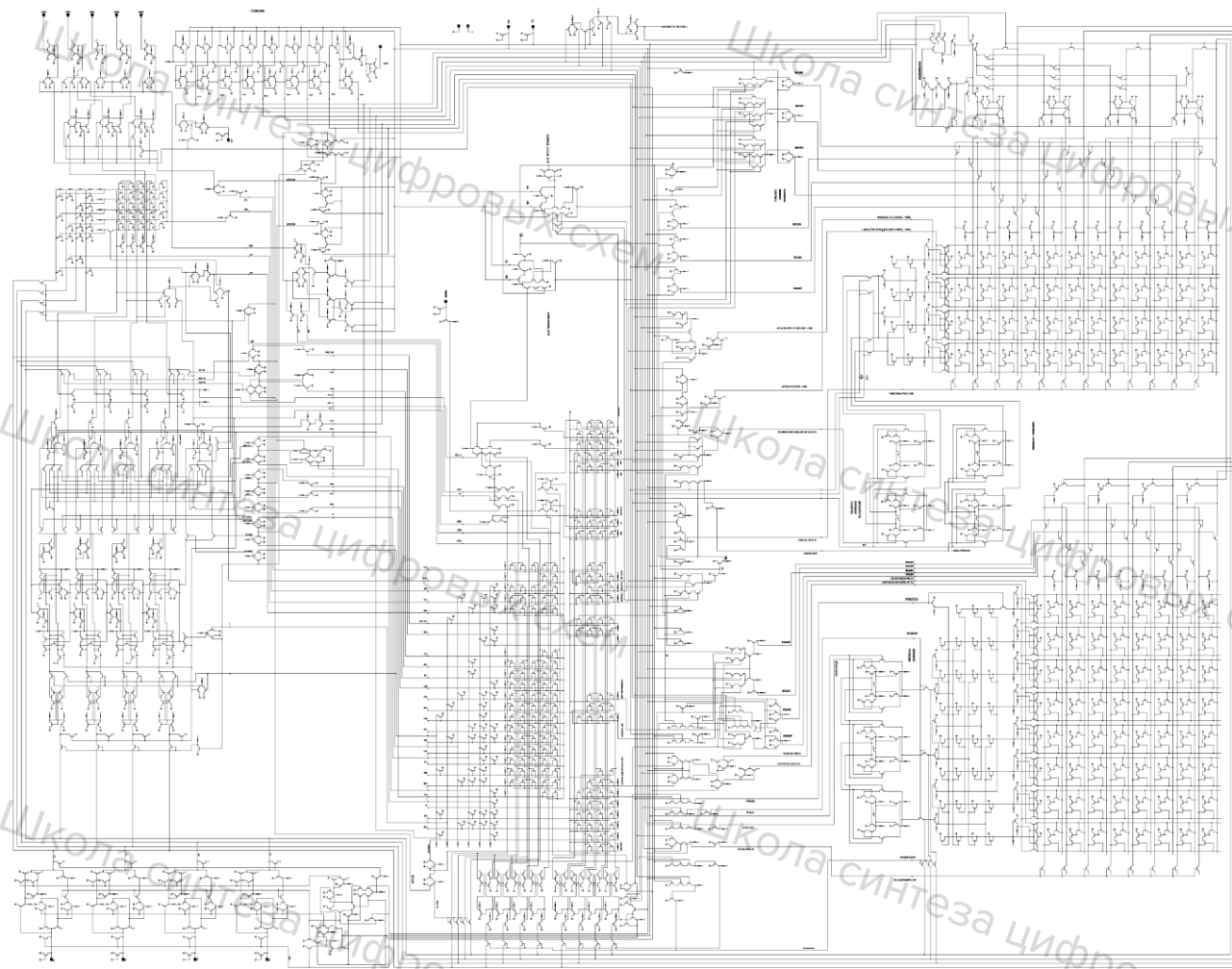


СХЕМА INTEL 4004



Intel 4004 Architecture

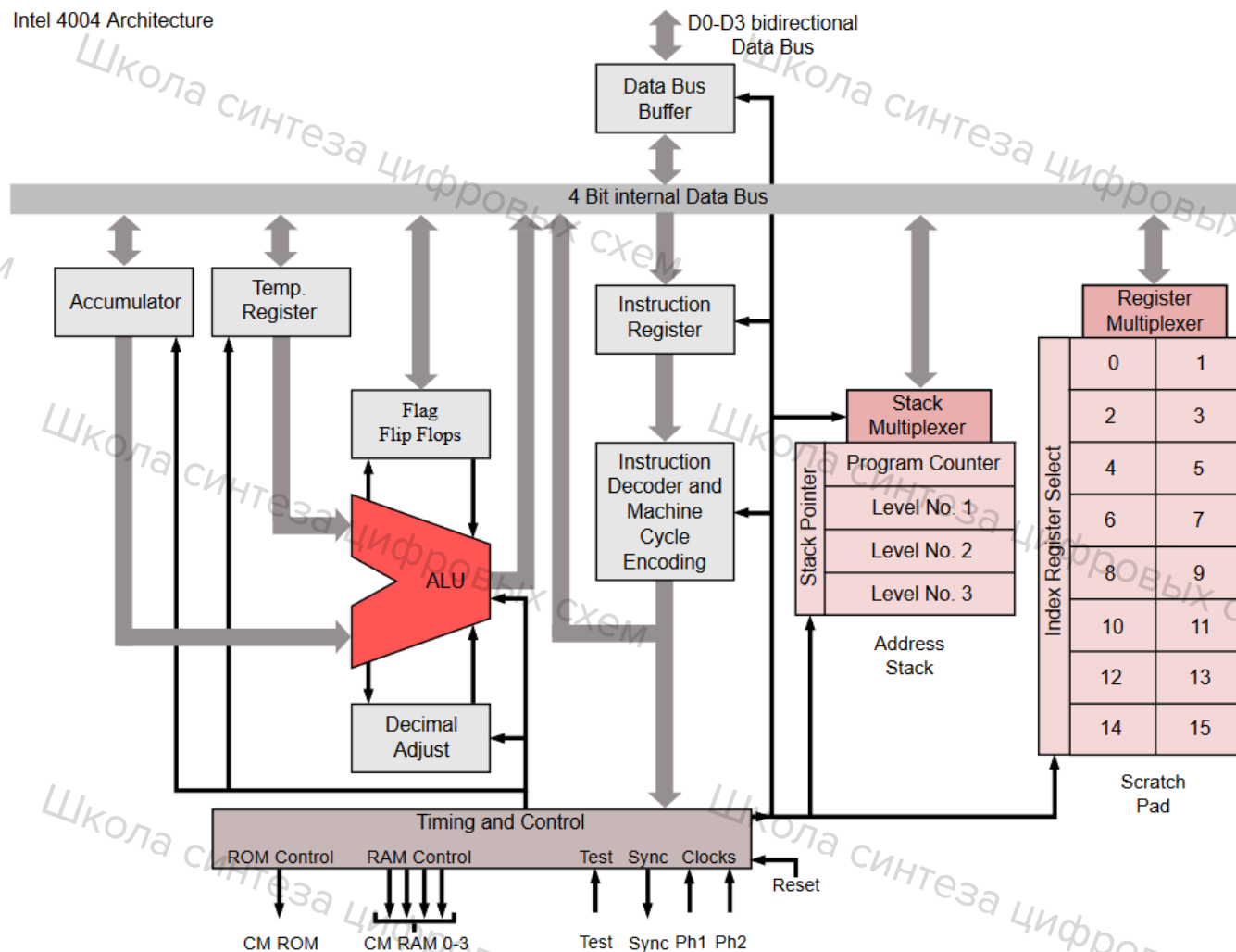
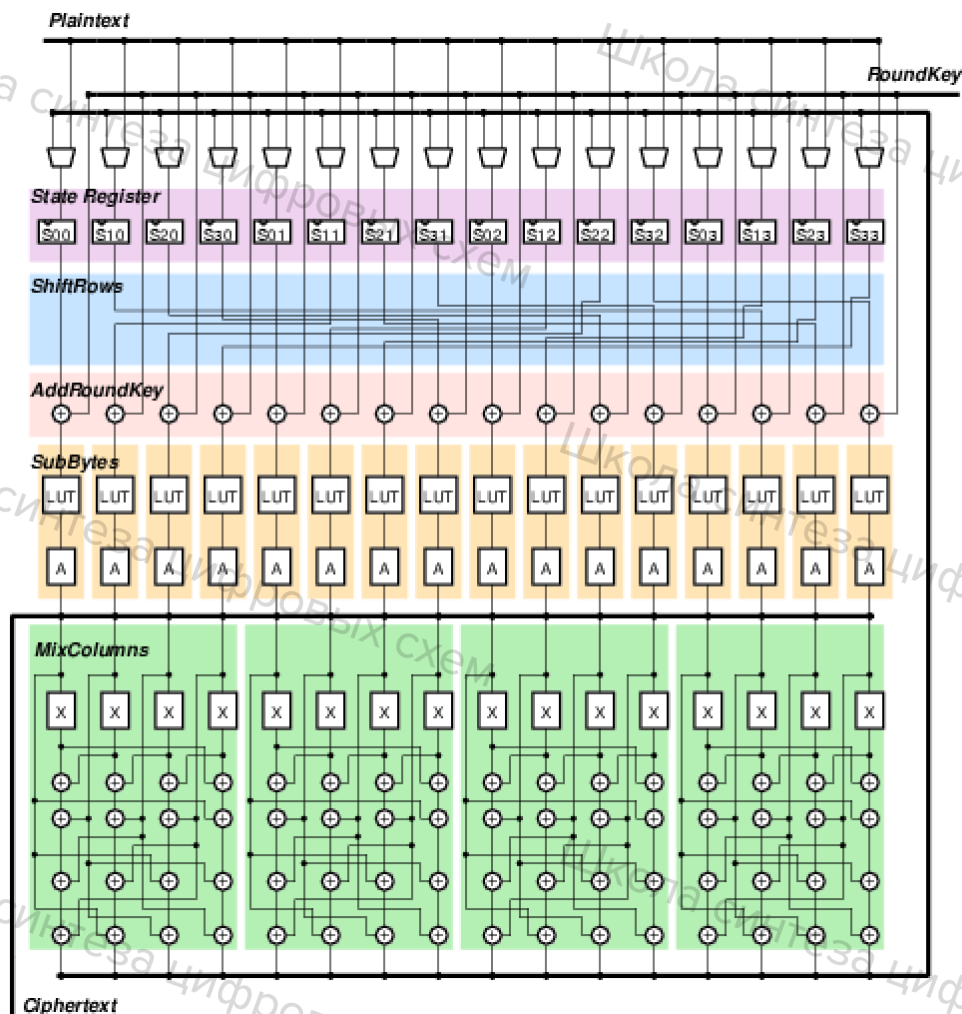


СХЕМА БЛОКА АППАРАТНОГО ШИФРОВАНИЯ AES



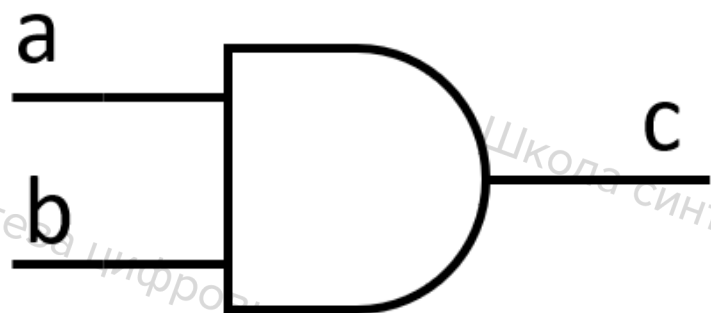
УПРАВЛЕНИЕ СЛОЖНОСТЬЮ В МИКРОЭЛЕКТРОНИКЕ



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ



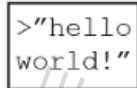
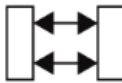
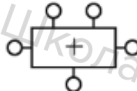
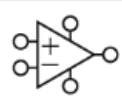
HDL — HARDWARE DESCRIPTION LANGUAGE



`assign c = a & b;`



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ

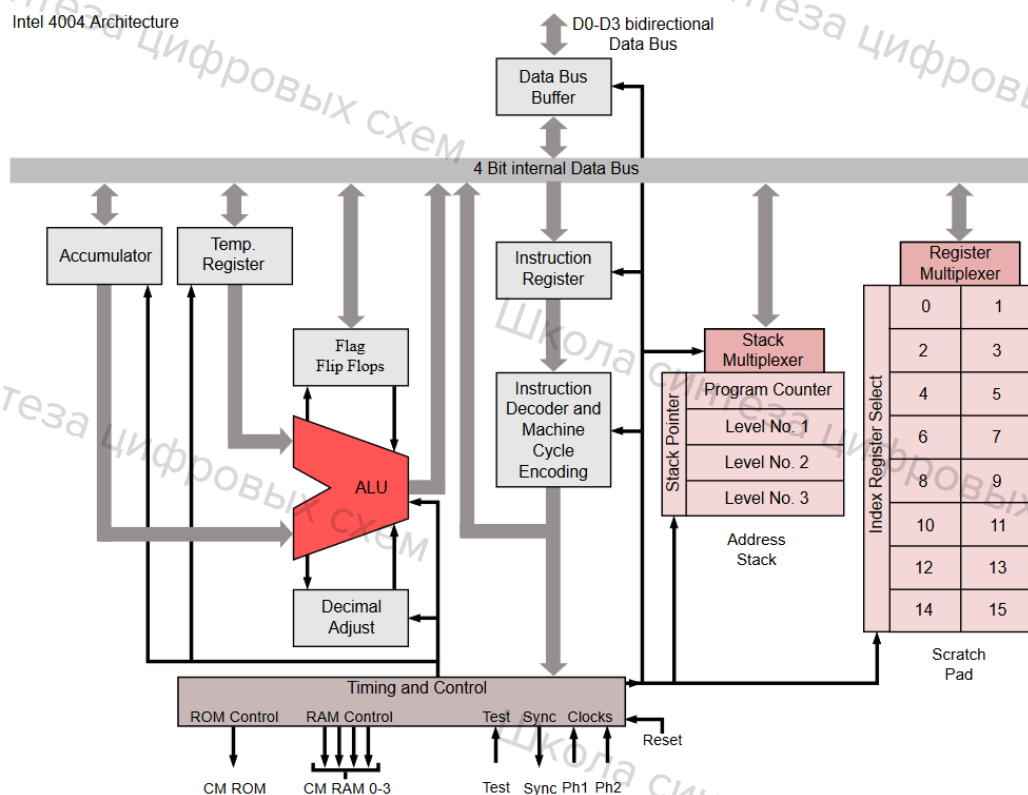
Application Software		Programs
Operating Systems		Device Drivers
Architecture		Instructions Registers
Micro-architecture		Datapaths Controllers
Logic		Adders Memories
Digital Circuits		AND Gates NOT Gates
Analog Circuits		Amplifiers Filters
Devices		Transistors Diodes
Physics		Electrons

HDL — НЕ ЯЗЫК ПРОГРАММИРОВАНИЯ



Intel 4004

Intel 4004 Architecture



И. К. Айвазовский Девятый вал



VERILOG HDL. ИСТОРИЯ

- Verilog был разработан компанией Gateway Design Automation в 1984 году как фирменный язык для симуляции логических схем.
- В 1989 году Gateway приобрела компания Cadence, и Verilog стал открытым стандартом в 1990 году под управлением сообщества Open Verilog International.
- Язык стал стандартом IEEE в 1995 году.
- В 2005 году язык был расширен для упорядочивания и лучшей поддержки моделирования и верификации систем.
- Эти расширения были объединены в единый стандарт, который сейчас называется SystemVerilog(стандарт IEEE 1800-2009).



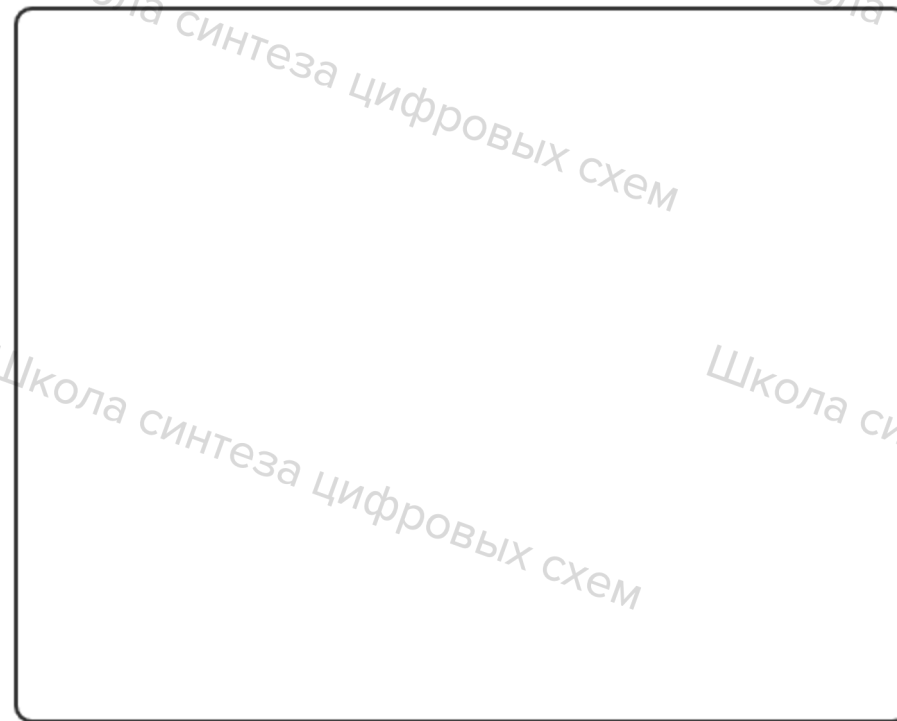
Клик правой кнопкой —
изменить изображение

SYSTEMVERILOG HDL



module

endmodule



SYSTEMVERILOG HDL



```
module top (
```

```
endmodule
```



SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);
```

```
endmodule
```

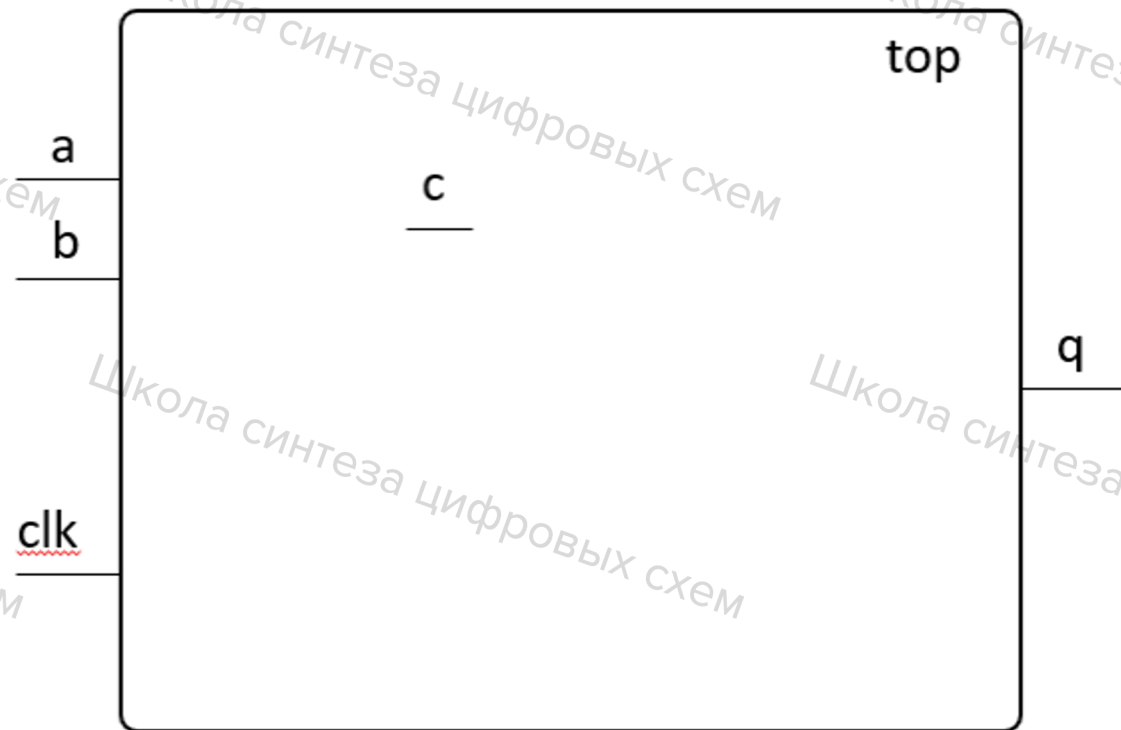


SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic c;
```

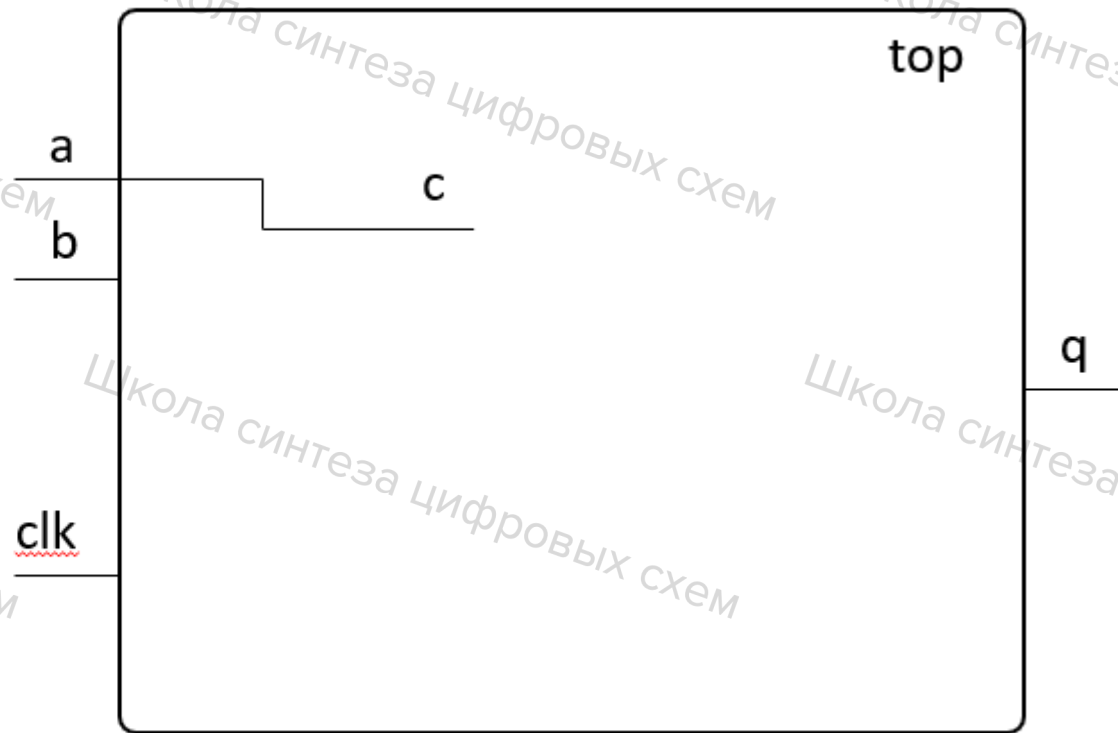
```
endmodule
```



SYSTEMVERILOG HDL



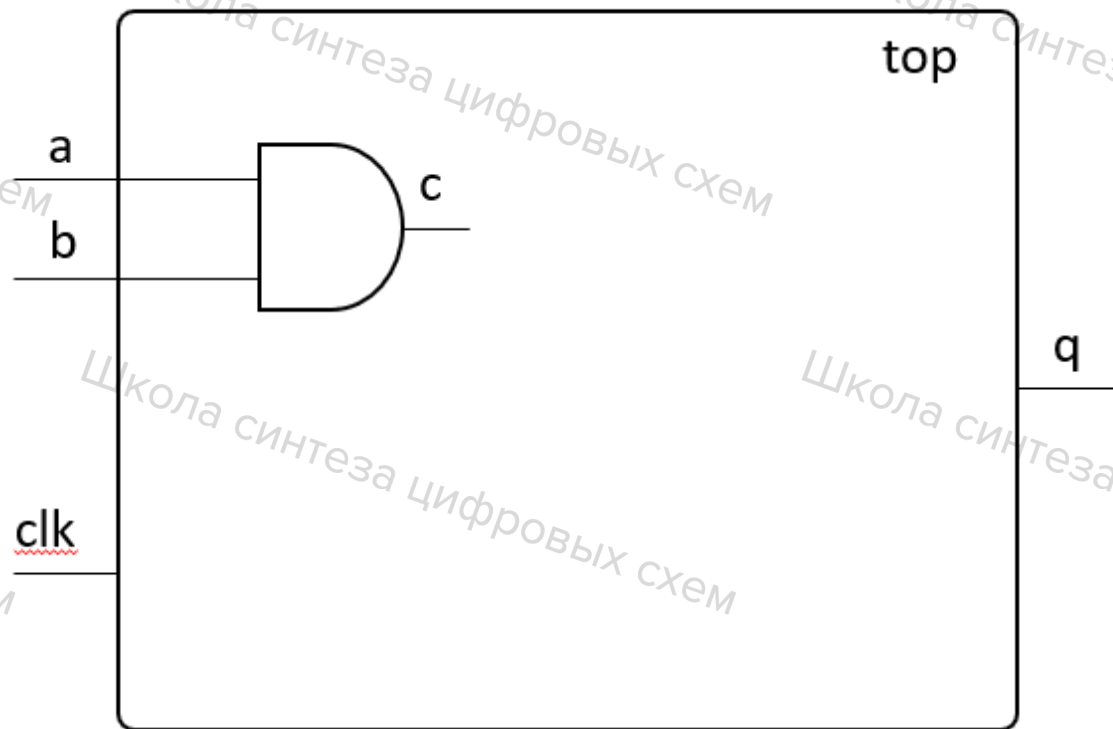
```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic c;  
  
    assign c = a;  
endmodule
```



SYSTEMVERILOG HDL



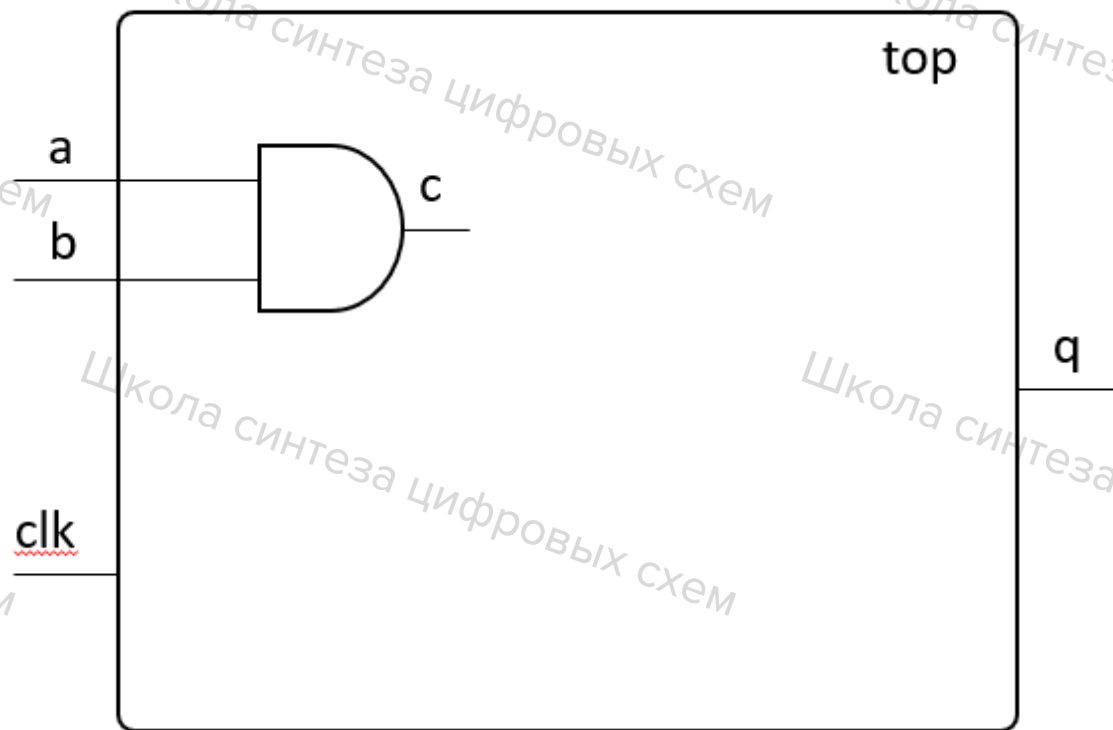
```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic c;  
  
    assign c = a & b;  
endmodule
```



SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic c;  
    // это комментарий  
    assign c = a & b;  
endmodule
```



SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic [3:0] c; // 4-х проводная шина  
    logic [3:0] d;  
    assign c = d;  
endmodule
```

SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);
```

```
    logic [3:0] c; d; // Объявление нескольких проводов одинаковой разрядности
```

```
    assign c = d;  
endmodule
```

SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic [3:0] c;  
    logic [8:0] d;  
    assign q = d[3]; // можно присвоить нужный бит  
endmodule
```

SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic [3:0] c;  
    logic [8:0] d;  
    assign q = d[6:3]; // можно присвоить диапазон бит  
endmodule
```


SYSTEMVERILOG HDL



```
module top (  
    input  clk,  
    input  a,  
    input  b,  
    output q  
);  
    logic [3:0] c;  
    logic [15:0] d;  
    assign q = d[c]; // можно назначить номер бита, определяемый шиной  
endmodule
```

SYSTEMVERILOG HDL



```
module top (  
    input      clk,  
    input [7:0] a,    // входные и выходные порты могут быть многоразрядными шинами  
    input      b,  
    output [3:0] q  
);  
  
    assign q = a[7:4];  
endmodule
```

SYSTEMVERILOG HDL



```
module top (  
    input      clk,  
    input [9:0] a,  
    input      b,  
    output [3:0] q  
);  
    logic [3:0] c;  
    logic [8:0] d [0:24]; // двумерный массив проводов  
    assign q = a[7:4];    // подключение 4-х бит из 10 шины массива  
endmodule
```



ФОРМАТ ОПИСАНИЯ ЧИСЕЛ SYSTEMVERILOG HDL

```
wire [10:0] a = 7;
```

```
// 32-х битное десятичное число, которое будет обрезано
```



ФОРМАТ ОПИСАНИЯ ЧИСЕЛ SYSTEMVERILOG HDL

```
wire [10:0] a = 7;  
wire [10:0] b = 'd7;
```

```
// 32-х битное десятичное число, которое будет обрезано  
// 11-ти битное десятичное число
```



ФОРМАТ ОПИСАНИЯ ЧИСЕЛ SYSTEMVERILOG HDL

```
wire [10:0] a = 7;           // 32-х битное десятичное число, которое будет обрезано
wire [10:0] b = 'd7;        // 11-ти битное десятичное число
wire [10:0] b = 11'd7;      // 11-ти битное десятичное число
```




ФОРМАТ ОПИСАНИЯ ЧИСЕЛ SYSTEMVERILOG HDL

```
wire [10:0] a = 7;           // 32-х битное десятичное число, которое будет обрезано
wire [10:0] b = 'd7;         // 11-ти битное десятичное число
wire [10:0] b = 11'd7;       // 11-ти битное десятичное число
wire [3: 0] c = 4'b0101;     // 4-х битное двоичное число
```



ФОРМАТ ОПИСАНИЯ ЧИСЕЛ SYSTEMVERILOG HDL

```
wire [10:0] a = 7;           // 32-х битное десятичное число, которое будет обрезано
wire [10:0] b = 'd7;        // 11-ти битное десятичное число
wire [10:0] b = 11'd7;      // 11-ти битное десятичное число
wire [3: 0] c = 4'b0101;    // 4-х битное двоичное число
wire [7: 0] d = 8'h7B;      // 8-ми битное шестнадцатеричное число 7B
```



ФОРМАТ ОПИСАНИЯ ЧИСЕЛ SYSTEMVERILOG HDL

```
wire [10:0] a = 7; // 32-х битное десятичное число, которое будет обрезано
wire [10:0] b = 'd7; // 11-ти битное десятичное число
wire [10:0] b = 11'd7; // 11-ти битное десятичное число
wire [3: 0] c = 4'b0101; // 4-х битное двоичное число
wire [7: 0] d = 8'h7B; // 8-ми битное шестнадцатеричное число 7B
wire [47:0] e = 48'hEFCA7ED98F; // 48-ми битное шестнадцатеричное число EFCA7ED98F
```



ФОРМАТ ОПИСАНИЯ ЧИСЕЛ SYSTEMVERILOG HDL

```
wire [10:0] a = 7; // 32-х битное десятичное число, которое будет обрезано
wire [10:0] b = 'd7; // 11-ти битное десятичное число
wire [10:0] b = 11'd7; // 11-ти битное десятичное число
wire [3: 0] c = 4'b0101; // 4-х битное двоичное число
wire [7: 0] d = 8'h7B; // 8-ми битное шестнадцатеричное число 7B
wire [47:0] e = 48'hEFCA7ED98F; // 48-ми битное шестнадцатеричное число EFCA7ED98F
wire signed [10:0] b = -11'd7; // 11-ти битное отрицательное десятичное число
```



ОСНОВНЫЕ ОПЕРАЦИИ VERILOG HDL

Символ	Назначение
{ }	Конкатенация(concatenation)
+ - * /	Арифметические(arithmetic)
%	Модуль(modulus)
> >= < <=	Отношения(relational)
!	Логическое отрицание(logical NOT)
&&	Логическое И(logical AND)
	Логическое ИЛИ(logical OR)



ОСНОВНЫЕ ОПЕРАЦИИ VERILOG HDL

Символ	Назначение
==	Логическое равенство(logical equality)
!=	Логическое неравенство(logical inequality)
===	Идентичность (case equality)
!==	Не идентичность (case inequality)
~	Побитовая инверсия (bit-wise NOT)
&	Побитовое И (bit-wise AND)
	Побитовое ИЛИ (bit-wise OR)



ОСНОВНЫЕ ОПЕРАЦИИ VERILOG HDL

Символ	Назначение
<<	Сдвиг влево (left shift)
>>	Сдвиг вправо (right shift)
<<<	Арифметический сдвиг влево
>>>	Арифметический сдвиг вправо
?:	Тернарный оператор (ternary)



ПОДГОТОВКА К ЗАПУСКУ УПРАЖНЕНИЙ

1. Необходимый софт и драйвера установлены согласно инструкции.
2. Скачайте репозиторий с примерами <https://github.com/chipdesignschool/basics-graphics-music>
3. Инструкции по загрузке репозитория и запуску примеров находятся в readme 
4. Подключите отладочную плату к компьютеру
5. Запустите скрипт [06_choose_another_fpga_board.bash](#) и выберете для сборки плату которая у вас на руках
6. Проверьте синтез и программирование первого примера [01_and_or_not_xor_de_morgan](#) запустив скрипт [03_synthesize_for_fpga.bash](#)

Getting Started

You only need a few things to get started:

1. Get a [compatible FPGA board](#).
2. Install [drivers and required software](#).
3. Download or checkout this repository on [Windows](#), [Linux](#), or [macOS](#).
4. Run `06_choose_another_fpga_board.bash` to select your board.
5. Run `03_synthesize_for_fpga.bash` to synthesize, place, and program your board all in one go.

That's it! You're now running Verilog example code on real hardware. Press buttons on the board and check for LEDs to light up accordingly.

First time setup

1. Follow the instructions for [Windows](#), [Linux](#), or [macOS](#) to set up your environment.
2. Optional: Configure VSCode:
 - i. Install these extensions:  *TODO: Verilog extensions configuration*
 - ii. Recommended extensions: *GitHub Pull Requests* and *GitHub Repositories*

ЗАПУСК УПРАЖНЕНИЙ

[basics-graphics-music](#) / labs / 



yuri-panchul

Some experiments with FPGA synthesis

Name



..



1_basics



2_graphics



3_music



4_microarchitecture



5_cpu/5_1_schoolriscv



6_misc/6_1_geiger_muller_radiation_counter



7_homework/7_1_serial_adder_using_logic_operations_only



8_unfinished



9_meetup_in_la



common



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ

ЗАПУСК УПРАЖНЕНИЙ

[basics-graphics-music](#) / labs /

yuri-panchul Some experiments with FPGA synthesis

Name



..



1_basics



2_graphics



3_music



4_microarchitecture



5_cpu/5_1_schoolriscv



6_misc/6_1_geiger_muller_radiation_counter



7_homework/7_1_serial_adder_using_logic_operations_only



8_unfinished



9_meetup_in_la



common

[basics-graphics-music](#) / labs / 1_basics /

yuri-panchul Added undeclared wire

Name



..



1_01_and_or_not_xor_de_morgan



1_02_mux



1_03_decoder



1_04_priority_encoder



1_05_7seven_segment_letter

ЗАПУСК УПРАЖНЕНИЙ

[basics-graphics-music](#) / labs / 



yuri-panchul Some experiments with FPGA synthesis

Name



..



1_basics



2_graphics



3_music



4_microarchitecture



5_cpu/5_1_schoolriscv



6_misc/6_1_geiger_muller_radiation_counter



7_homework/7_1_serial_adder_using_logic_operations_only



8_unfinished



9_meetup_in_la



common



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ

[basics-graphics-music](#) / labs / [2_graphics](#) /



yuri-panchul Fixed a minor bug

Name



..



2_1_rectangle_ellipse_parabola



2_2_rectangle_ellipse_parabola_moving



2_3_color_stripes



2_4_game

ЗАПУСК УПРАЖНЕНИЙ



- Для запуска проекта, синтеза и конфигурации запустите скрипт номер 03.
- В процессе запуска он спросит вас о плате под которую собрать пример.
- Выберите название платы которой вы пользуетесь.
- Если вы хотите запустить САПР с GUI выберите 04 скрипт
- Для смены платы под которую собирается проект используйте скрипт 06.

Name	Last commit message
..	
01_clean.bash	Renamed all .source.bash to .source_bash to avoi...
02_simulate_rtl.bash	Renamed all .source.bash to .source_bash to avoi...
03_synthesize_for_fpga.bash	Renamed all .source.bash to .source_bash to avoi...
04_configure_fpga.bash	Renamed all .source.bash to .source_bash to avoi...
05_run_gui_for_fpga_synthesis.bash	Did big renaming to accomodate a new script 05...
06_choose_another_fpga_board.bash	Did big renaming to accomodate a new script 05...
07_synthesize_for_asic.bash	Did big renaming to accomodate a new script 05...
08_visualize_asic_synthesis_results_1.ba...	Did big renaming to accomodate a new script 05...
09_visualize_asic_synthesis_results_2.ba...	Did big renaming to accomodate a new script 05...
tb.sv	Developing OpenLane synthesis script
top.sv	Fixed incompatibility with VCS (inout logic)

ЗАПУСК УПРАЖНЕНИЙ

- Исходный код примеров находится в файле **lab_top.sv** в каждой из директорий.



Name
..
01_clean.bash
02_simulate_rtl.bash
03_synthesize_for_fpga.bash
04_configure_fpga.bash
05_run_gui_for_fpga_synthesis.bash
06_choose_another_fpga_board.bash
07_synthesize_for_asic.bash
08_visualize_asic_synthesis_results_1.bash
09_visualize_asic_synthesis_results_2.bash
lab_top.sv
tb.sv

ЗАПУСК УПРАЖНЕНИЙ

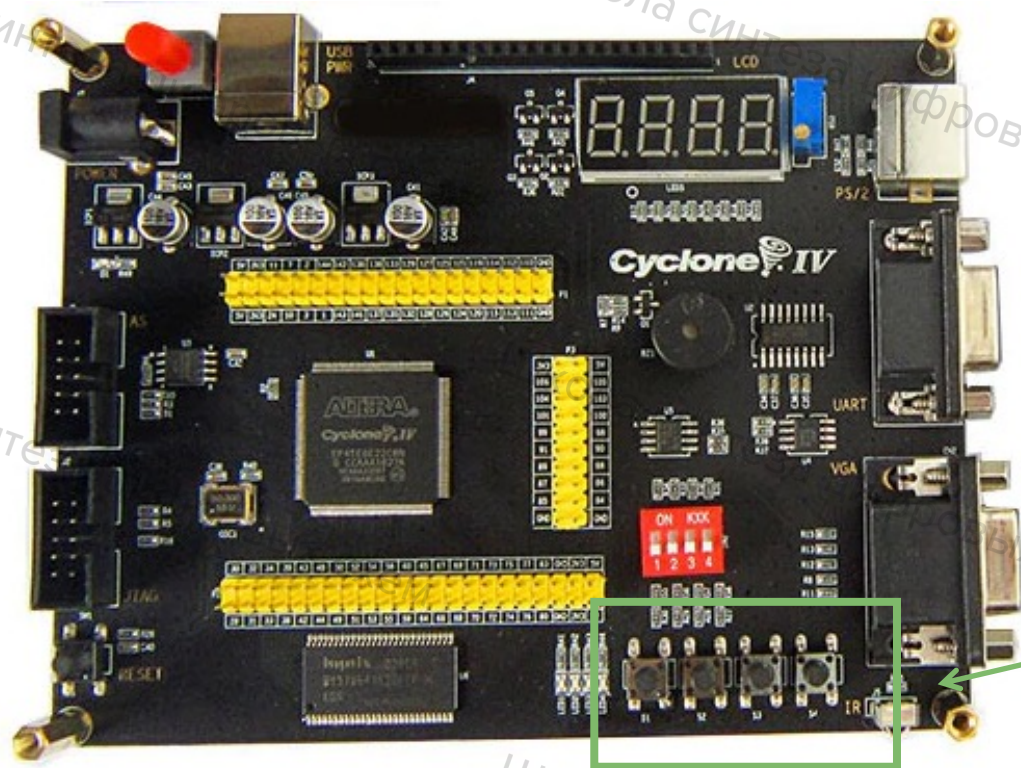


- Исходный код примеров находится в файле **lab_top.sv** в каждой из директорий.
- Задания, которые необходимо выполнить, написаны в нем же в комментариях.

```
98 //-----
99
100 // Exercise 1: Change the code below.
101 // Assign to led [2] the result of AND operation.
102 //
103 // If led [2] is not available on your board,
104 // comment out the code above and reuse led [0].
105
106 // assign led [2] =
107
108 // Exercise 2: Change the code below.
109 // Assign to led [3] the result of XOR operation
110 // without using "&" operation.
111 // Use only operations "&", "|", "~" and parenthesis, "(" and ")".
112
113 // assign led [3] =
114
115 // Exercise 3: Create an illustration to De Morgan's laws:
116 //
117 // ~ (a & b) == ~ a | ~ b
118 // ~ (a | b) == ~ a & ~ b
119
120 endmodule
```

Name
..
01_clean.bash
02_simulate_rtl.bash
03_synthesize_for_fpga.bash
04_configure_fpga.bash
05_run_gui_for_fpga_synthesis.bash
06_choose_another_fpga_board.bash
07_synthesize_for_asic.bash
08_visualize_asic_synthesis_results_1.bash
09_visualize_asic_synthesis_results_2.bash
lab_top.sv
tb.sv

УПРАЖНЕНИЕ С ЛОГИЧЕСКИМИ ЭЛЕМЕНТАМИ 1_01_and_or_not_xor_de_morgan



Вывод результатов логических операций над входными воздействиями с кнопок на светодиоды.



УПРАЖНЕНИЕ

С ЛОГИЧЕСКИМИ ЭЛЕМЕНТАМИ

1_01_and_or_not_xor_de_morgan

```

3  module top
4      #
5      parameter clk_mhz = 50,
6              w_key   = 4,
7              w_sw    = 8,
8              w_led   = 8,
9              w_digit = 8,
10             w_gpio  = 20
11  )
12  (
13      input          clk,
14      input          rst,
15
16      // Keys, switches, LEDs
17
18      input [w_key - 1:0] key,
19      input [w_sw - 1:0] sw,
20      output logic [w_led - 1:0] led
21
22  )
23
24  //-----
25
26  wire a = key [0];
27  wire b = key [1];
28
29  wire result = a ^ b;
30
31  assign led [0] = result;
32
33  assign led [1] = key [0] ^ key [1];
34
35  //-----
36
37  // Exercise 1: Change the code below.
38  // Assign to led [2] the result of AND operation.
39  // If led [2] is not available on your board,
40  // comment out the code above and reuse led [0].
41
42  // assign led [2] =
43
44  // Exercise 2: Change the code below.
45  // Assign to led [3] the result of XOR operation
46  // without using "^" operation.
47  // Use only operations "&", "|", "~" and parenthesis "(" and ")".
48
49  // assign led [3] =
50
51  // Exercise 3: Create an illustration to De Morgan's laws:
52  //
53  // ~ (a & b) == ~ a | ~ b
54  // ~ (a | b) == ~ a & ~ b
55
56  //-----
57
58  // Exercise 4: Create an illustration to De Morgan's laws:
59  //
60  // ~ (a & b) == ~ a | ~ b
61  // ~ (a | b) == ~ a & ~ b
62
63  //-----

```

```

77  //-----
78
79  // Exercise 1: Change the code below.
80  // Assign to led [2] the result of AND operation.
81  // If led [2] is not available on your board,
82  // comment out the code above and reuse led [0].
83
84  // assign led [2] =
85
86  // Exercise 2: Change the code below.
87  // Assign to led [3] the result of XOR operation
88  // without using "^" operation.
89  // Use only operations "&", "|", "~" and parenthesis "(" and ")".
90
91  // assign led [3] =
92
93  // Exercise 3: Create an illustration to De Morgan's laws:
94  //
95  // ~ (a & b) == ~ a | ~ b
96  // ~ (a | b) == ~ a & ~ b
97
98  //-----

```



УПРАЖНЕНИЕ

С ЛОГИЧЕСКИМИ ЭЛЕМЕНТАМИ

1_01_and_or_not_xor_de_morgan

```

3  module top
4  # (
5      parameter clk_mhz = 50,
6          w_key   = 4,
7          w_sw    = 8,
8          w_led   = 8,
9          w_digit = 8,
10         w_gpio  = 20
11 )
12 (
13     input      clk,
14     input      rst,
15
16     // Keys, switches, LEDs
17
18     input  [w_key - 1:0] key,
19     input  [w_sw  - 1:0] sw,
20     output logic [w_led - 1:0] led,
21
22     //-----
23
24     // Exercise 1: Change the code below.
25     // Assign to led [2] the result of AND operation.
26     // If led [2] is not available on your board,
27     // comment out the code above and reuse led [0].
28
29     // assign led [2] =
30
31     // Exercise 2: Change the code below.
32     // Assign to led [3] the result of XOR operation
33     // without using "^" operation.
34     // Use only operations "&", "|", "~" and parenthesis, "(" and ")".
35
36     // assign led [3] =
37
38     // Exercise 3: Create an illustration to De Morgan's laws:
39     //
40     // ~ (a & b) == ~ a | ~ b
41     // ~ (a | b) == ~ a & ~ b
42
43     wire a = key [0];
44     wire b = key [1];
45
46     wire result = a ^ b;
47
48     assign led [0] = result;
49     assign led [1] = key [0] ^ key [1];
50
51     //-----
52
53
54
55
56
57
58
59
60
61
62
63

```

```

77 //-----
78
79 // Exercise 1: Change the code below.
80 // Assign to led [2] the result of AND operation.
81 // If led [2] is not available on your board,
82 // comment out the code above and reuse led [0].
83
84 // assign led [2] =
85
86 // Exercise 2: Change the code below.
87 // Assign to led [3] the result of XOR operation
88 // without using "^" operation.
89 // Use only operations "&", "|", "~" and parenthesis, "(" and ")".
90
91 // assign led [3] =
92
93 // Exercise 3: Create an illustration to De Morgan's laws:
94 //
95 // ~ (a & b) == ~ a | ~ b
96 // ~ (a | b) == ~ a & ~ b
97
98

```



УПРАЖНЕНИЕ

С ЛОГИЧЕСКИМИ ЭЛЕМЕНТАМИ

1_01_and_or_not_xor_de_morgan

```

3  module top
4  # (
5      parameter clk_mhz = 50,
6          w_key   = 4,
7          w_sw    = 8,
8          w_led   = 8,
9          w_digit = 8,
10         w_gpio  = 20
11 )
12 (
13     input      clk,
14     input      rst,
15
16     // Keys, switches, LEDs
17
18     input  [w_key  - 1:0] key,
19     input  [w_sw   - 1:0] sw,
20     output logic [w_led - 1:0] led,
21
22     //-----
23
24     wire a = key [0];
25     wire b = key [1];
26
27     wire result = a ^ b;
28
29     assign led [0] = result;
30
31     assign led [1] = key [0] ^ key [1];
32
33

```

```

77     //-----
78
79     // Exercise 1: Change the code below.
80     // Assign to led [2] the result of AND operation.
81     //
82     // If led [2] is not available on your board,
83     // comment out the code above and reuse led [0].
84
85     // assign led [2] =
86
87     // Exercise 2: Change the code below.
88     // Assign to led [3] the result of XOR operation
89     // without using "^" operation.
90     // Use only operations "&", "|", "~", and parenthesis, "(" and ")".
91
92     // assign led [3] =
93
94     // Exercise 3: Create an illustration to De Morgan's laws:
95     //
96     // ~ (a & b) == ~ a | ~ b
97     // ~ (a | b) == ~ a & ~ b
98

```



УПРАЖНЕНИЕ

С ЛОГИЧЕСКИМИ ЭЛЕМЕНТАМИ

1_01_and_or_not_xor_de_morgan

Символ**Назначение**

==

Логическое равенство(logical equality)

!=

Логическое неравенство(logical inequality)

===

Идентичность (case equality)

!==

Не идентичность (case inequality)

~

Побитовая инверсия (bit-wise NOT)

&

Побитовое И (bit-wise AND)

|

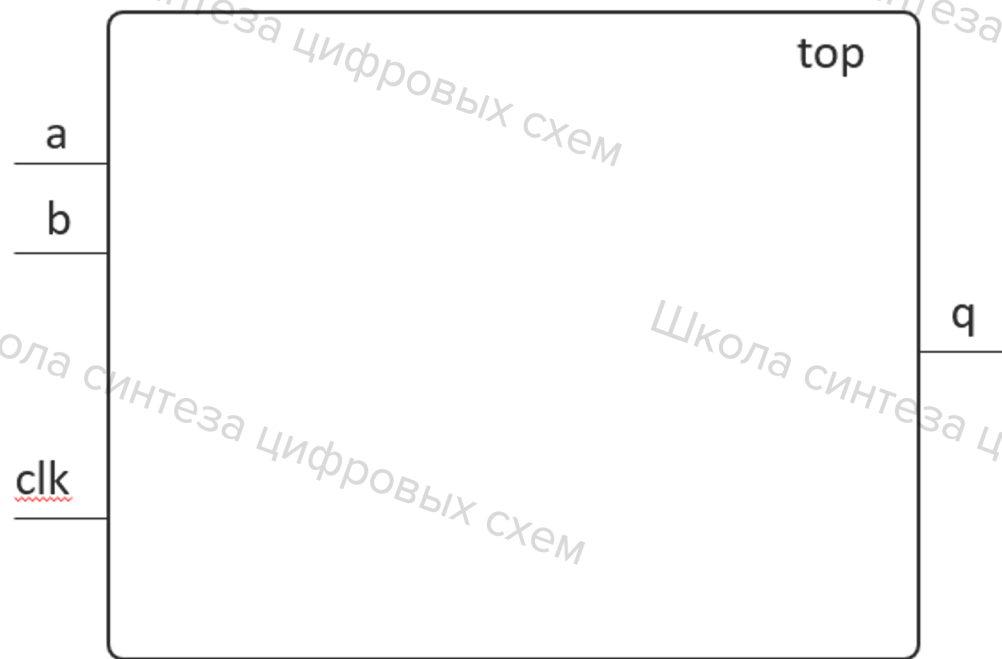
Побитовое ИЛИ (bit-wise OR)

МАНИПУЛЯЦИИ С БИТАМИ SYSTEMVERILOG HDL



```
module top (  
    input    clk,  
    input    a,  
    input    b,  
    output   q  
);
```

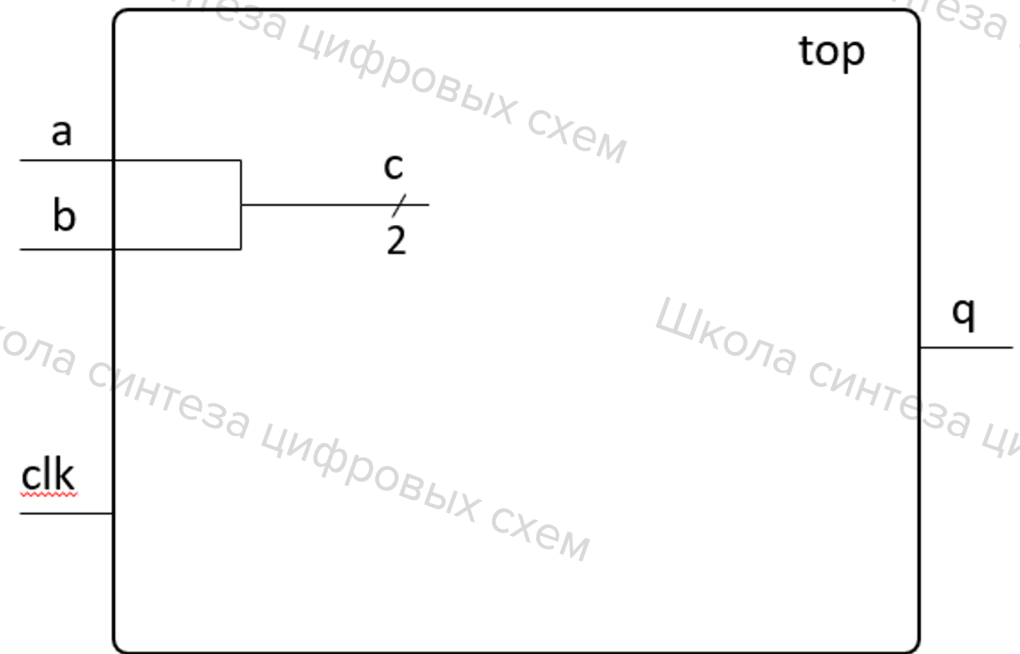
```
endmodule
```



МАНИПУЛЯЦИИ С БИТАМИ SYSTEMVERILOG HDL



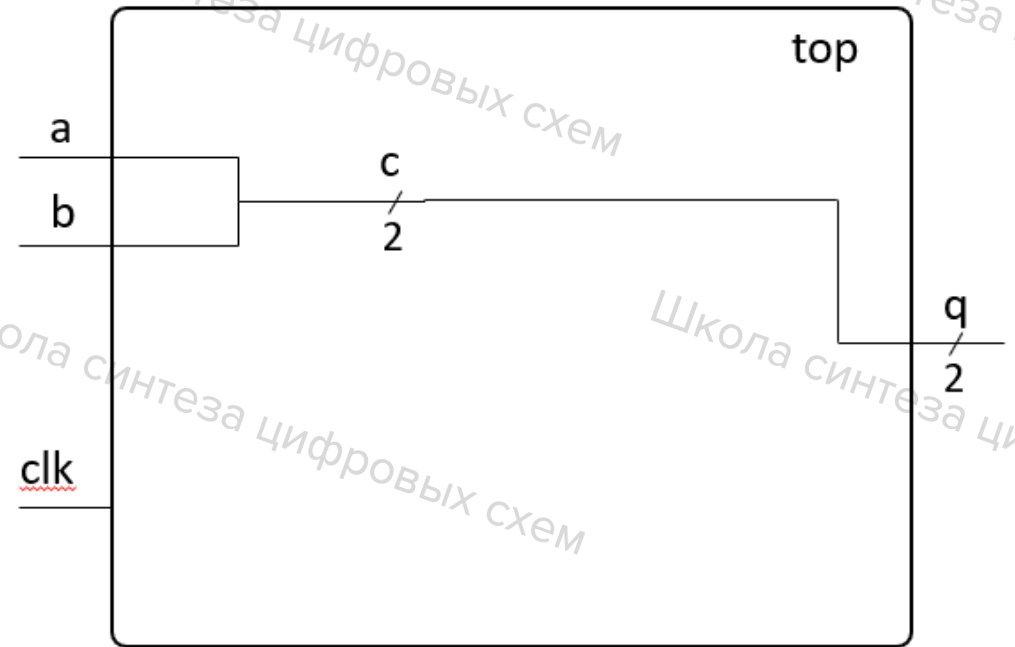
```
module top (  
    input    clk,  
    input    a,  
    input    b,  
    output   q  
);  
    logic [1:0] c; // многобитный сигнал(шин  
    assign c = {a, b}; // Конкатенация  
  
endmodule
```



МАНИПУЛЯЦИИ С БИТАМИ SYSTEMVERILOG HDL



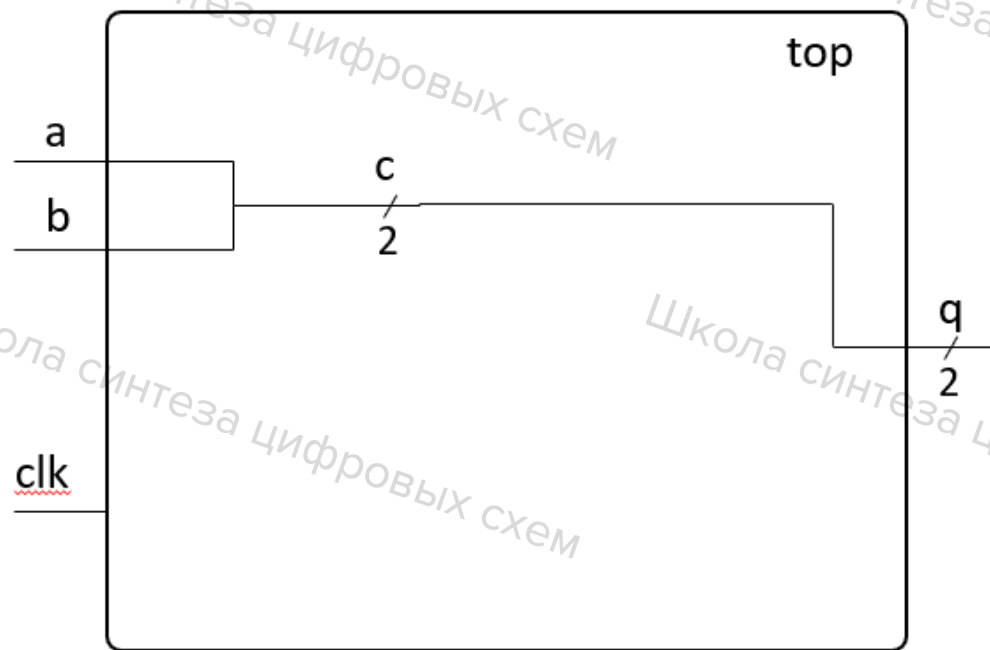
```
module top (  
    input    clk,  
    input    a,  
    input    b,  
    output   q  
);  
    logic [1:0] c; // многобитный сигнал(шина)  
    assign c = {a, b}; // Конкатенация  
    assign q = c[0]; // Обращение к биту  
  
endmodule
```



МАНИПУЛЯЦИИ С БИТАМИ SYSTEMVERILOG HDL



```
module top (  
    input      clk,  
    input      a,  
    input      b,  
    output [1:0] q  
);  
    logic [1:0] c; // многобитный сигнал(шина)  
    assign c = {a, b}; // Конкатенация  
    assign q = c;  
  
endmodule
```



SYSTEMVERILOG HDL. БЛОК ALWAYS



```
module top (  
    input                clk,  
    input [7:0] a,  
    input               b,  
    output logic [3:0] q // входные и выходные сигналы и шины можно объявлять как logic  
);  
    always @(*) begin  
        q = ~a[7:4];  
    end  
endmodule
```

SYSTEMVERILOG HDL. БЛОК ALWAYS



```
module top (  
    input          clk,  
    input [7:0]    a,  
    input          b,  
    output logic [3:0] q // входные и выходные сигналы и шины можно объявлять как logic  
);  
    always @(*) begin  
        q = !a[7:4];  
    end  
endmodule
```

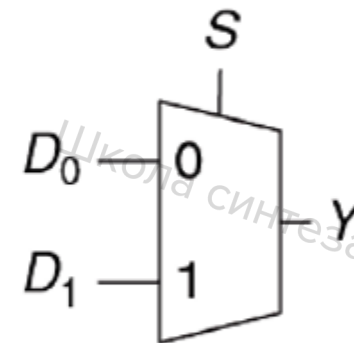
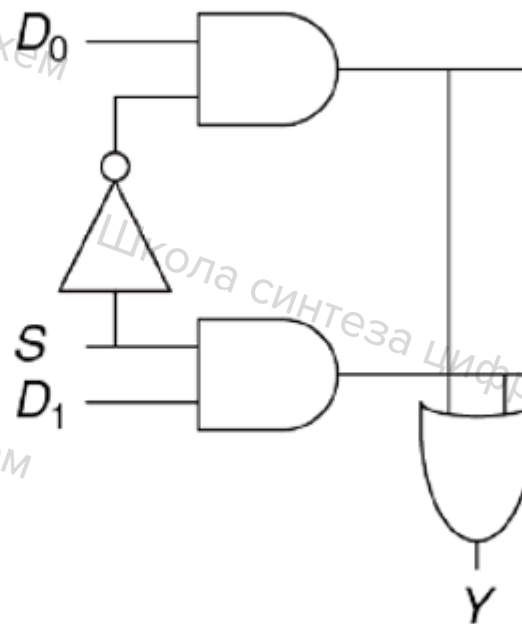
SYSTEMVERILOG HDL. МУЛЬТИПЛЕКСОР



```
module mux(  
    input  D0,  
    input  D1,  
    input  S,  
    output Y  
);
```

```
    assign Y = S ? D1 : D0;
```

```
endmodule
```





SYSTEMVERILOG HDL.

БЛОК ALWAYS

// Связанные между собой превратятся в общую цепь комбинационной логики

```
logic [3:0] a, b, c, d, e;
```

```
logic [3:0] f, g, h, j;
```

```
always_comb begin
```

```
    f = a + b;
```

```
    g = f & c;
```

```
    h = g | d;
```

```
    j = h - e;
```

```
end
```




SYSTEMVERILOG HDL.

БЛОК ALWAYS

// Связанные между собой превратятся в общую цепь комбинационной логики

```
logic [3:0] a, b, c, d, e;
```

```
logic [3:0] f, g, h, j;
```

```
always_comb begin
```

```
    j = (((a + b) & c) | d) - e;
```

```
end
```



SYSTEMVERILOG HDL.

IF-ELSE

// можно описывать мультиплексоры с помощью if-else, а не тернарного оператора

```
logic [3:0] c;  
always_comb begin  
    if (d) begin  
        c = a & b;  
    end  
    else begin  
        c = a | b;  
    end  
end
```



SYSTEMVERILOG HDL. CASE

// многовходовые мультиплексоры и дешифраторы можно описывать через case

```
logic [1:0] option;  
logic [7:0] a, b, c, d;  
logic [7:0] e;
```

```
always_comb begin
```

```
    case (option)
```

```
        0: e = a;
```

```
        1: e = b;
```

```
        2: e = c;
```

```
        3: e = d;
```

```
    endcase
```

```
end
```



SYSTEMVERILOG HDL. CASE

// многовходовые мультиплексоры и дешифраторы можно описывать через case

```
logic [1:0] option;  
logic [7:0] a, b, c, d;  
logic [7:0] e;
```

```
always_comb begin  
    case (option)  
        0: e = a;  
        1: e = b;  
        default: e = d;  
    endcase  
end
```



УПРАЖНЕНИЕ С МУЛЬТИПЛЕКСОРАМИ

1_02_MUX

```
53 //-----  
54  
55 wire sel, a, b;  
56  
57 generate  
58     if (w_key >= 3)  
59         begin : use_keys  
60             assign sel = key [2];  
61             assign a  = key [1];  
62             assign b  = key [0];  
63         end  
64     else  
65         begin : use_switches  
66             assign sel = sw [2];  
67             assign a  = sw [1];  
68             assign b  = sw [0];  
69         end  
70     endgenerate  
71
```



УПРАЖНЕНИЕ С МУЛЬТИПЛЕКСОРАМИ

1_02_MUX

```

72 //-----
73
74 // Five different implementations
75
76 wire mux0 = sel ? a : b;
77
78 //-----
79
80 wire [1:0] ab = { a, b };
81 assign mux1 = ab [sel];
82
83 //-----
84
85 logic mux2;
86
87 always_comb
88     if (sel)
89         mux2 = a;
90     else
91         mux2 = b;
92
93 //-----
94
95 logic mux3;
96
97 always_comb
98     case (sel)
99         1'b1: mux3 = a;
100        1'b0: mux3 = b;
101     endcase
102

```

```

103 //-----
104
105 // Exercise 1: Implement mux
106 // without using "?" operation, "if", "case" or a bit selection.
107 // Use only operations "&", "|", "~" and parenthesis, "(" and ")".
108
109 wire mux4 = 1'b0;
110
111 //-----
112
113 // Use table
114
115 wire [0:7] table5 =
116 {
117     1'b0, // sel = 0, a = 0, b = 0
118     1'b1, // sel = 0, a = 0, b = 1
119     1'b0, // sel = 0, a = 1, b = 0
120     1'b1, // sel = 0, a = 1, b = 1
121     1'b0, // sel = 1, a = 0, b = 0
122     1'b0, // sel = 1, a = 0, b = 1
123     1'b1, // sel = 1, a = 1, b = 0
124     1'b1 // sel = 1, a = 1, b = 1
125 };
126
127 wire mux5 = table5 [{ sel, a, b }];
128
129 // Exercise 2: Change the table to get the correct result by doing
130 // wire mux5_2 = table5_2 [{ a, b, sel }];
131

```



УПРАЖНЕНИЕ С МУЛЬТИПЛЕКСОРАМИ

1_02_MUX

```

231 //-----
232
233 `ifdef SYNOPSIS_CADENCE_MENTOR
234
235 // This syntax probably works only with Synopsys VCS, Cadence Xselium
236 // and QuestaSim from Siemens EDA (former Mentor Graphics)
237
238 logic table11 [0:1][0:1][0:1] =
239 '{
240     '{
241         '{ 1'b0, 1'b0 }, // a = 0, b = 0, sel = 0/1
242         '{ 1'b1, 1'b0 } // a = 0, b = 1, sel = 0/1
243     },
244     '{
245         '{ 1'b1, 1'b0 }, // a = 1, b = 0, sel = 0/1
246         '{ 1'b1, 1'b1 } // a = 1, b = 1, sel = 0/1
247     }
248 };
249
250
251 wire mux11 = table11 [a][b][sel];
252
253 // Exercise 3: Change the table to get the correct result by doing
254 // wire mux11_2 = table11_2 [sel][b][a];
255
256 `else
257
258 wire mux11 = mux0;
259
260 `endif
261

```

```

262 //-----
263
264 // Use concatenation operation for all signals:
265
266 wire [11:0] all_muxes
267 = { mux11 , mux10 , mux9 , mux8 ,
268     mux7 , mux6 , mux5 , mux4 ,
269     mux3 , mux2 , mux1 , mux0 };
270
271 assign led = w_led' (all_muxes);
272
273 // Use concatenation operation for the boards with 4 LEDs:
274
275 // assign led = w_led' ({ mux3 , mux2 , mux1 , mux0 });
276 // assign led = w_led' ({ mux6 , mux5 , mux4 , mux0 });
277 // assign led = w_led' ({ mux9 , mux8 , mux7 , mux0 });
278 // assign led = w_led' ({ mux11 , mux10 , mux4 , mux0 });
279

```




SYSTEMVERILOG HDL.

СЛОЖЕНИЕ И ВЫЧИТАНИЕ

```
module simple_add_sub(  
    input  [7:0] operandA, operandB, // два входных 8-ми битных операнда  
    output [8:0] out_sum, out_dif    // Выходы для арифметических операций имеют  
    // дополнительный 9-й бит переполнения  
);  
  
// Максимальное 8-ми битное беззнаковое число 255. При сложении 255 + 255 получится  
// 510, которое можно представить минимум 9-ю битами  
assign out_sum = operandA + operandB;  
assign out_dif = operandA - operandB;  
// Если сделать выход сумматора 8-ми битным, то случится переполнение и результат  
// сложения 255 + 3 будет равен 2  
endmodule
```



SYSTEMVERILOG HDL.

ОПЕРАЦИИ НАД ЗНАКОВЫМИ ДАННЫМИ

```
module simple_add_sub(  
    input  [7:0] operandA, operandB, // два входных 8-ми битных операнда  
    output [8:0] out_sum, out_dif    // Выходы для арифметических операций имеют  
    // дополнительный 9-й бит переполнения  
);  
// Для корректного выполнения знаковых операций операнды и результат должны быть  
// объявлены как signed  
// Минимальное 8-ми битное знаковое число -128, а максимальное – 127. При сложении -128  
// + 127 получится 1  
assign out_sum = operandA + operandB;  
assign out_dif = operandA - operandB;  
endmodule
```



SYSTEMVERILOG HDL.

УМНОЖЕНИЕ И ДЕЛЕНИЕ

```
module simple_mul_div(  
    input  [7:0] operandA, operandB, // два входных восьмибитных операнда  
    output [15:0] out_mul, out_div, out_rem // Для умножения, чтобы избежать  
    переполнения, выходная разрядность должна быть не меньше чем сумма разрядностей  
    операндов для беззнакового случая и -1 для знакового. Для данного случая это 16 и 15  
    бит. Для знакового случая разрядность меньше из-за наличия старшего знакового бита.  
  
);  
  
assign out_mul = operandA * operandB; // Умножение  $8 * 8 = 64$   
assign out_div = operandA / operandB; // Деление  $16 / 3 = 5$ . Округление до целого числа  
    в меньшую сторону  
assign out_rem = operandA % operandB; // Остаток от деления  $16 \% 3 = 1$   
  
// ВАЖНО: Операции деления и остатка от деления не используются в синтезируемом коде –  
    только для симуляции. Так как дает очень сложную реализацию по площади и очень плохие  
    тайминги. Например, в процессорах деление реализуется итерационными алгоритмами  
endmodule
```



SYSTEMVERILOG HDL.

ЛОГИЧЕСКИЕ И АРИФМЕТИЧЕСКИЕ СДВИГИ

```
module simple_shifts (  
    input  [7:0] operandA, operandB, // два входных 8-ми битных операнда  
    output [7:0] out_sll, out_srl, out_sar // Выходы для операций сдвига  
);  
  
//логический сдвиг влево на значение operandB  
assign out_sll = operandA << operandB;  
  
// Число, на которое нужно сдвигать, определяется 3-мя битами второго операнда.  
// Например, operandA = 8'b1010_1110, operandB = 8'b0000_0011, тогда out_slr =  
// 8'b0001_0101  
assign out_slr = operandA >> operandB[2:0];  
  
// арифметический сдвиг вправо (сохранение знака числа). Например, operandA =  
// 8'b1111_1100, тогда out_sar = 8'b1111_1111  
assign out_sar = operandA >>> 3;  
endmodule
```



SYSTEMVERILOG HDL.

БИТОВЫЕ ЛОГИЧЕСКИЕ ОПЕРАЦИИ

```
module simple_logic_op (  
    input  [7:0] operandA, operandB, // два входных 8-ми битных операнда  
    output [7:0] out_bit_and, out_bit_or, out_bit_xor, out_bit_not  
);  
  
    assign out_bit_and = operandA & operandB; // 8'b0011_1001 & 8'b1110_0101 =  
    8'b0010_0001  
    assign out_bit_or  = operandA | operandB; // 8'b0011_1001 | 8'b1110_0101 =  
    8'b1111_1101  
    assign out_bit_xor = operandA ^ operandB; // 8'b0011_1001 & 8'b1110_0101 =  
    8'b1101_1100  
    assign out_bit_not = ~operandA;           // ~8'b0011_1001 = 8'b1100_0110  
  
endmodule
```



SYSTEMVERILOG HDL.

БУЛЕВЫЕ ЛОГИЧЕСКИЕ ОПЕРАЦИИ

```
module simple_bool_logic (  
    input [7:0] operandA, operandB,  
    output      out_bool_and, out_bool_or, out_bool_not  
);  
  
    assign out_bool_and = operandA && operandB; //8'b0011_1101 && 8'b1010_0110 = 1'b1  
    assign out_bool_or  = operandA || operandB; //8'b0011_1101 || 8'b1010_0110 = 1'b1  
    assign out_bool_not = !operandA;           //!8'b0011_1101 = 1'b0  
endmodule
```

SYSTEMVERILOG HDL. ОПЕРАЦИИ СВЕРТКИ



```
module simple_conv (  
    input [7:0] operandA,  
    output out_reduction_and, out_reduction_or, out_reduction_xor  
);
```

```
// Операции выполняются между битами одной шины
```

```
assign out_reduction_and = &operandA; // &8'b0110_1011 = 1'b0
```

```
assign out_reduction_or = |operandA; // |8'b0110_1011 = 1'b1
```

```
assign out_reduction_xor = ^operandA; // ^8'b0110_1011 = 1'b1
```

```
endmodule
```




SYSTEMVERILOG HDL.

ОПЕРАЦИИ СРАВНЕНИЯ

```
module simple_comp(  
    input [7:0] operandA, operandB,  
    output out_eq, out_ne, out_gt, out_lt, out_ge, out_le  
);
```

```
    assign out_eq = operandA == operandB;  
    assign out_ne = operandA != operandB;  
    assign out_ge = operandA >= operandB;  
    assign out_le = operandA <= operandB;  
    assign out_gt = operandA > operandB;  
    assign out_lt = operandA < operandB;
```

```
endmodule
```



УПРАЖНЕНИЕ С ДЕКОДЕРАМИ

1_03_DECODER

```

53 //-----
54
55 wire [1:0] in = { key [1], key [0] };
56
57 //-----
58
59 // Implementation 1: tedious
60
61 wire [3:0] dec0;
62
63 assign dec0 [0] = ~ in [1] & ~ in [0];
64 assign dec0 [1] = ~ in [1] & in [0];
65 assign dec0 [2] = in [1] & ~ in [0];
66 assign dec0 [3] = in [1] & in [0];
67
68 // Implementation 2: case
69
70 logic [3:0] dec1;
71
72 always_comb
73     case (in)
74         2'b00: dec1 = 4'b0001;
75         2'b01: dec1 = 4'b0010;
76         2'b10: dec1 = 4'b0100;
77         2'b11: dec1 = 4'b1000;
78     endcase
79
80 // Implementation 3: shift
81
82 wire [3:0] dec2 = 4'b0001 << in;
83
84 // Implementation 4: index
85
86 logic [3:0] dec3;
87
88 always_comb
89     begin
90         dec3 = '0;
91         dec3 [in] = 1'b1;
92     end
93
94 //-----
95
96 assign led = w_led' ({ dec0, dec1, dec2, dec3 });

```

УПРАЖНЕНИЕ С ДЕКОДЕРАМИ

1_03_DECODER



```

53 //-----
54
55 wire [1:0] in = { key [1], key [0] };
56
57 //-----
58
59 // Implementation 1: tedious
60
61 wire [3:0] dec0;
62
63 assign dec0 [0] = ~ in [1] & ~ in [0];
64 assign dec0 [1] = ~ in [1] & in [0];
65 assign dec0 [2] = in [1] & ~ in [0];
66 assign dec0 [3] = in [1] & in [0];
67
68 // Implementation 2: case
69
70 logic [3:0] dec1;
71
72 always_comb
73   case (in)
74     2'b00: dec1 = 4'b0001;
75     2'b01: dec1 = 4'b0010;
76     2'b10: dec1 = 4'b0100;
77     2'b11: dec1 = 4'b1000;
78   endcase
79
80 // Implementation 3: shift
81
82 wire [3:0] dec2 = 4'b0001 << in;
83
84 // Implementation 4: index
85
86 logic [3:0] dec3;
87
88 always_comb
89   begin
90     dec3 = '0;
91     dec3 [in] = 1'b1;
92   end
93
94 //-----
95
96 assign led = w_led' ({ dec0, dec1, dec2, dec3 });

```

УПРАЖНЕНИЕ С ДЕКОДЕРАМИ

1_03_DECODER



```

53 //-----
54
55 wire [1:0] in = { key [1], key [0] };
56
57 //-----
58
59 // Implementation 1: tedious
60
61 wire [3:0] dec0;
62
63 assign dec0 [0] = ~ in [1] & ~ in [0];
64 assign dec0 [1] = ~ in [1] & in [0];
65 assign dec0 [2] = in [1] & ~ in [0];
66 assign dec0 [3] = in [1] & in [0];
67
68 // Implementation 2: case
69
70 logic [3:0] dec1;
71
72 always_comb
73     case (in)
74         2'b00: dec1 = 4'b0001;
75         2'b01: dec1 = 4'b0010;
76         2'b10: dec1 = 4'b0100;
77         2'b11: dec1 = 4'b1000;
78     endcase
79

```

```

80 // Implementation 3: shift
81
82 wire [3:0] dec2 = 4'b0001 << in;
83
84 // Implementation 4: index
85
86 logic [3:0] dec3;
87
88 always_comb
89     begin
90         dec3 = '0;
91         dec3 [in] = 1'b1;
92     end
93
94 //-----
95
96 assign led = w_led' ({ dec0, dec1, dec2, dec3 });

```

УПРАЖНЕНИЕ С ДЕКОДЕРАМИ

1_03_DECODER



```

53 //-----
54
55 wire [1:0] in = { key [1], key [0] };
56
57 //-----
58
59 // Implementation 1: tedious
60
61 wire [3:0] dec0;
62
63 assign dec0 [0] = ~ in [1] & ~ in [0];
64 assign dec0 [1] = ~ in [1] & in [0];
65 assign dec0 [2] = in [1] & ~ in [0];
66 assign dec0 [3] = in [1] & in [0];
67
68 // Implementation 2: case
69
70 logic [3:0] dec1;
71
72 always_comb
73     case (in)
74         2'b00: dec1 = 4'b0001;
75         2'b01: dec1 = 4'b0010;
76         2'b10: dec1 = 4'b0100;
77         2'b11: dec1 = 4'b1000;
78     endcase
79
80 // Implementation 3: shift
81
82 wire [3:0] dec2 = 4'b0001 << in;
83
84 // Implementation 4: index
85
86 logic [3:0] dec3;
87
88 always_comb
89     begin
90         dec3 = '0;
91         dec3 [in] = 1'b1;
92     end
93
94 //-----
95
96 assign led = ~led' ({ dec0, dec1, dec2, dec3 });

```



УПРАЖНЕНИЕ С ЕНКОДЕРАМИ

1_04_PRIORITY_ENCODER

```

53  //-----
54
55  wire [2:0] in;
56
57  generate
58
59      if (w_key >= 3)                // Board with at least 3 keys
60          assign in = key [2:0];
61      else if (w_sw >= 3)            // Board with at least 3 switches
62          assign in = sw [2:0];
63      else if (w_key + w_sw >= 3)    // Board with at least 3 keys + switches
64          assign in = 3'({ key, sw });
65      else if (w_key >= 2)           // Board with at least 2 keys
66          assign in = { key [0], key [1:0] };
67      else                           // Corner case: repeat a key 3 times
68          assign in = { 3 { key [0] } };
69
70  endgenerate
71

```

```

72  //-----
73
74  logic [1:0] enc0, enc1, enc2, enc3;
75
76  // Implementation 1. Priority encoder using a chain of "ifs"
77
78  always_comb
79      if (in [0]) enc0 = 2'd0;
80      else if (in [1]) enc0 = 2'd1;
81      else if (in [2]) enc0 = 2'd2;
82      else
83          enc0 = 2'd0;
84
85  // Implementation 2. Priority encoder using casez
86
87  always_comb
88      casez (in)
89          3'b??1: enc1 = 2'd0;
90          3'b?10: enc1 = 2'd1;
91          3'b100: enc1 = 2'd2;
92          default: enc1 = 2'd0;
93      endcase

```

УПРАЖНЕНИЕ С ЕНКОДЕРАМИ

1_04_PRIORITY_ENCODER



```

53 //-----
54
55 wire [2:0] in;
56
57 generate
58
59     if (w_key >= 3)                // Board with at least 3 keys
60         assign in = key [2:0];
61     else if (w_sw >= 3)            // Board with at least 3 switches
62         assign in = sw [2:0];
63     else if (w_key + w_sw >= 3)    // Board with at least 3 keys + switches
64         assign in = 3' ( { key, sw } );
65     else if (w_key >= 2)           // Board with at least 2 keys
66         assign in = { key [0], key [1:0] };
67     else                          // Corner case: repeat a key 3 times
68         assign in = { 3 { key [0] } };
69
70 endgenerate
71

```

```

72 //-----
73
74 logic [1:0] enc0, enc1, enc2, enc3;
75
76 // Implementation 1. Priority encoder using a chain of "ifs"
77
78 always_comb
79     if (in [0]) enc0 = 2'd0;
80     else if (in [1]) enc0 = 2'd1;
81     else if (in [2]) enc0 = 2'd2;
82     else
83         enc0 = 2'd0;
84
85 // Implementation 2. Priority encoder using casez
86
87 always_comb
88     casez (in)
89         3'b??1: enc1 = 2'd0;
90         3'b?10: enc1 = 2'd1;
91         3'b100: enc1 = 2'd2;
92         default: enc1 = 2'd0;
93     endcase
94

```


УПРАЖНЕНИЕ С ЕНКОДЕРАМИ

1_04_PRIORITY_ENCODER



```

94 // Implementation 3: Combination of priority arbiter
95 // and encoder without priority
96
97 localparam w = 3;
98
99 wire [w - 1:0] c = { ~ in [w - 2:0] & c [w - 2:0], 1'b1 };
100 wire [w - 1:0] g = in & c;
101
102 always_comb
103     unique case (g)
104         3'b001: enc2 = 2'd0;
105         3'b010: enc2 = 2'd1;
106         3'b100: enc2 = 2'd2;
107         default: enc2 = 2'd0;
108     endcase
109
110 /*
111 // A variation of Implementation 3: Using unusual case of "case"
112
113 always_comb
114     unique case (1'b1)
115         g [0]: enc2 = 2'd0;
116         g [1]: enc2 = 2'd1;
117         g [2]: enc2 = 2'd2;
118         default: enc2 = 2'd0;
119     endcase
120 */
121

```

```

122 // A note on obsolete practice:
123 //
124 // Before the SystemVerilog construct "unique case"
125 // got supported by the synthesis tools,
126 // the designers were using pseudo-comment "synopsys parallel_case":
127 //
128 // SystemVerilog : unique case (1'b1)
129 // Verilog 2001 : case (1'b1) // synopsys parallel_case
130
131 // Implementation 4: Using "for" loop
132
133 `ifdef __ICARUS__
134
135     // Icarus does not support break statement
136
137     assign enc3 = enc2;
138 `else
139
140     always_comb
141     begin
142         enc3 = '0;
143
144         for (int i = 0; i < $bits (in); i++)
145         begin
146             if (in [i])
147             begin
148                 enc3 = 2' (i);
149                 break;
150             end
151         end
152     end
153
154 `endif
155
156
157 //-----
158
159 assign led = w_led' ({ enc0, enc1, enc2, enc3 });

```

УПРАЖНЕНИЕ С ЕНКОДЕРАМИ

1_04_PRIORITY_ENCODER



```

94 // Implementation 3: Combination of priority arbiter
95 // and encoder without priority
96
97 localparam w = 3;
98
99 wire [w - 1:0] c = { ~ in [w - 2:0] & c [w - 2:0], 1'b1 };
100 wire [w - 1:0] g = in & c;
101
102 always_comb
103     unique case (g)
104         3'b001: enc2 = 2'd0;
105         3'b010: enc2 = 2'd1;
106         3'b100: enc2 = 2'd2;
107         default: enc2 = 2'd0;
108     endcase
109
110 /*
111 // A variation of Implementation 3: Using unusual case of "case"
112
113 always_comb
114     unique case (1'b1)
115         g [0]: enc2 = 2'd0;
116         g [1]: enc2 = 2'd1;
117         g [2]: enc2 = 2'd2;
118         default: enc2 = 2'd0;
119     endcase
120 */
121

```

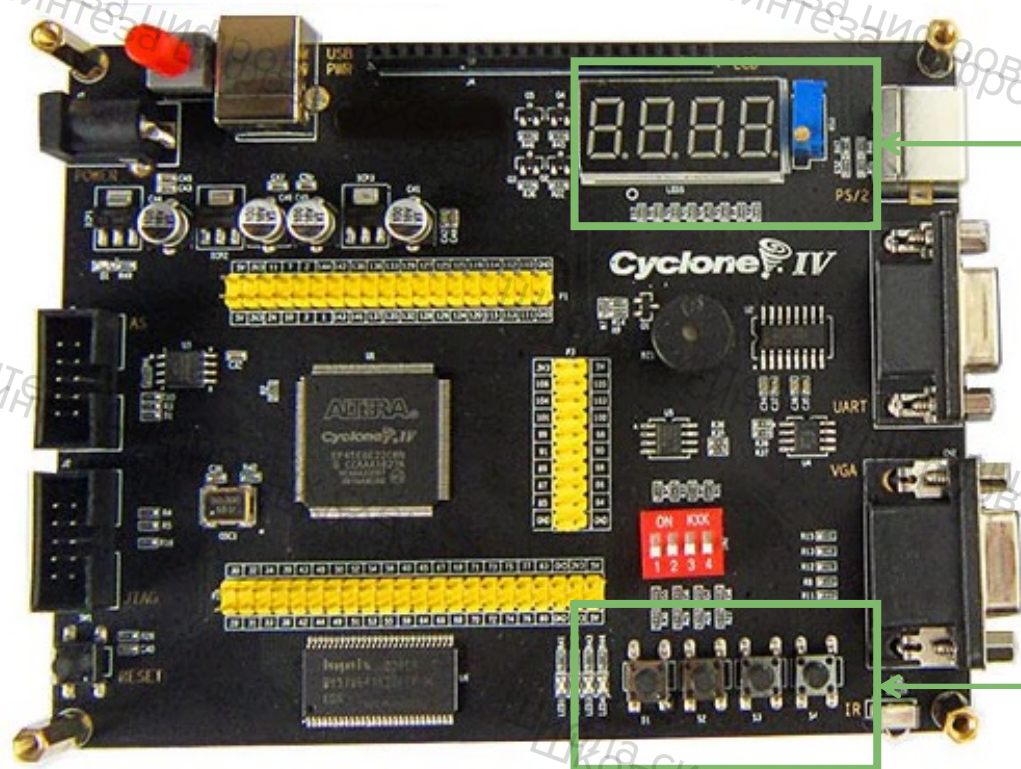
```

122 // A note on obsolete practice:
123 //
124 // Before the SystemVerilog construct "unique case"
125 // got supported by the synthesis tools,
126 // the designers were using pseudo-comment "synopsys parallel_case":
127 //
128 // SystemVerilog : unique case (1'b1)
129 // Verilog 2001 : case (1'b1) // synopsys parallel_case
130
131 // Implementation 4: Using "for" loop
132
133 `ifdef __ICARUS__
134
135     // Icarus does not support break statement
136
137     assign enc3 = enc2;
138 `else
139
140
141     always_comb
142     begin
143         enc3 = '0;
144
145         for (int i = 0; i < $bits (in); i++)
146         begin
147             if (in [i])
148             begin
149                 enc3 = 2' (i);
150                 break;
151             end
152         end
153     end
154
155 `endif
156
157 //-----
158
159 assign led = w_led' ({ enc0, enc1, enc2, enc3 });

```



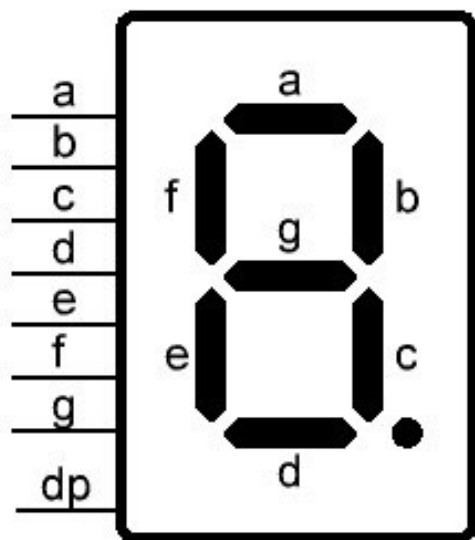
УПРАЖНЕНИЕ С ВЫВОДОМ БУКВЫ НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_05_7seven_segment_letter



Вывод статичных значений на семисегментный индикатор. Взаимодействие с семисегментным индикатором через кнопки.



СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР



Dec	4-bit шина				7-сегментный индикатор						
	3	2	1	0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1



УПРАЖНЕНИЕ С СЕМИСЕГМЕНТНЫМ ИНДИКАТОРОМ 1_05_7seven_segment_letter

```

53 //-----
54
55 //  --a--
56 // |      |
57 // f      b
58 // |      |
59 //  --g--
60 // |      |
61 // e      c
62 // |      |
63 //  --d-- h
64
65 typedef enum bit [7:0]
66 {
67     F = 8'b1000_1110,
68     P = 8'b1100_1110,
69     G = 8'b1011_1100,
70     A = 8'b1110_1110,
71     space = 8'b0000_0000
72 }
73 seven_seg_encoding_e;
74
75 assign abcdefgh = key [0] ? P : F;
76 assign digit     = w_digit' (key [1] ? 2'b10 : 2'b01);

```

```

78 // Exercise 1: Display the first letters
79 // of your first name and last name instead.
80
81 // assign abcdefgh = ...
82 // assign digit     = ...
83
84 // Exercise 2: Display letters of a 4-character word
85 // using this code to display letter of FPGA as an example
86
87 /*
88     seven_seg_encoding_e letter;
89
90     always_comb
91     case (4'(key))
92         4'b1000: letter = F;
93         4'b0100: letter = P;
94         4'b0010: letter = G;
95         4'b0001: letter = A;
96         default: letter = space;
97     endcase
98
99     assign abcdefgh = letter;
100     assign digit     = w_digit' (key);
101 */
102

```




УПРАЖНЕНИЕ С СЕМИСЕГМЕНТНЫМ ИНДИКАТОРОМ 1_05_7seven_segment_letter

```

53 //-----
54
55 //  --a--
56 // |      |
57 // f      b
58 // |      |
59 //  --g--
60 // |      |
61 // e      c
62 // |      |
63 //  --d-- h
64
65 typedef enum bit [7:0]
66 {
67     F = 8'b1000_1110,
68     P = 8'b1100_1110,
69     G = 8'b1011_1100,
70     A = 8'b1110_1110,
71     space = 8'b0000_0000
72 }
73 seven_seg_encoding_e;
74
75 assign abcdefgh = key [0] ? P : F;
76 assign digit     = w_digit' (key [1] ? 2'b10 : 2'b01);

```

```

78 // Exercise 1: Display the first letters
79 // of your first name and last name instead.
80
81 // assign abcdefgh = ...
82 // assign digit     = ...
83
84 // Exercise 2: Display letters of a 4-character word
85 // using this code to display letter of FPGA as an example
86
87 /*
88     seven_seg_encoding_e letter;
89
90     always_comb
91     case (4'(key))
92         4'b1000: letter = F;
93         4'b0100: letter = P;
94         4'b0010: letter = G;
95         4'b0001: letter = A;
96         default: letter = space;
97     endcase
98
99     assign abcdefgh = letter;
100     assign digit     = w_digit' (key);
101 */
102

```



УПРАЖНЕНИЕ С СЕМИСЕГМЕНТНЫМ ИНДИКАТОРОМ 1_05_7seven_segment_letter

```

53 //-----
54
55 //  --a--
56 //  |    |
57 //  f    b
58 //  |    |
59 //  --g--
60 //  |    |
61 //  e    c
62 //  |    |
63 //  --d-- h
64
65 typedef enum bit [7:0]
66 {
67     F = 8'b1000_1110,
68     P = 8'b1100_1110,
69     G = 8'b1011_1100,
70     A = 8'b1110_1110,
71     space = 8'b0000_0000
72 }
73 seven_seg_encoding_e;
74
75 assign abcdefgh = key [0] ? P : F;
76 assign digit    = w_digit' (key [1] ? 2'b10 : 2'b01);

```

```

78 // Exercise 1: Display the first letters
79 // of your first name and last name instead.
80
81 // assign abcdefgh = ...
82 // assign digit    = ...
83
84 // Exercise 2: Display letters of a 4-character word
85 // using this code to display letter of FPGA as an example
86

```

```

87 /*
88 seven_seg_encoding_e letter;
89
90 always_comb
91 case (4'(key))
92 4'b1000: letter = F;
93 4'b0100: letter = P;
94 4'b0010: letter = G;
95 4'b0001: letter = A;
96 default: letter = space;
97 endcase
98
99 assign abcdefgh = letter;
100 assign digit    = w_digit' (key);
101 */
102

```

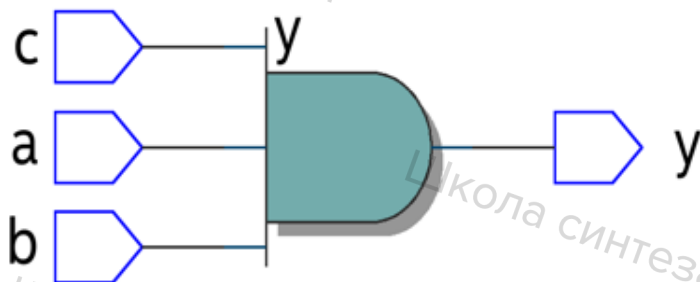

SYSTEMVERILOG HDL. ИЕРАРХИЯ МОДУЛЕЙ



```
module and_3 (
    input  a, b, c,
    output y
);
```

```
assign y = a & b & c;
```

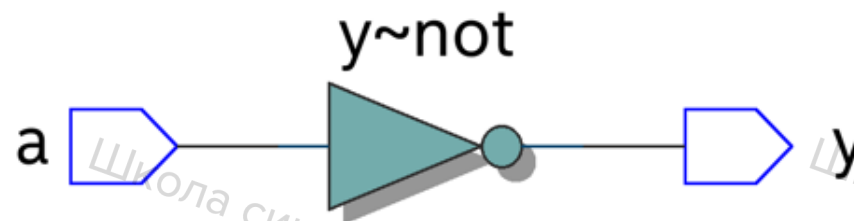
```
endmodule
```



```
module inv (
    input  a,
    output y
);
```

```
assign y = ~a;
```

```
endmodule
```





SYSTEMVERILOG HDL. ИЕРАРХИЯ МОДУЛЕЙ

```

module top (
    input      a, b, c,
    output    y
);

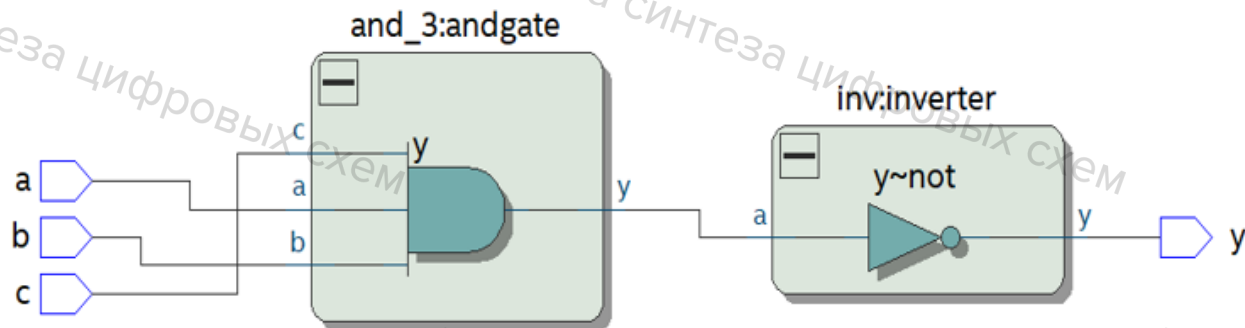
wire n1;

    and_3 andgate (
        .a(a), .b(b),
        .c(c), .y(n1)
    );

    inv inverter (
        .a(n1), .y(y)
    );

endmodule

```



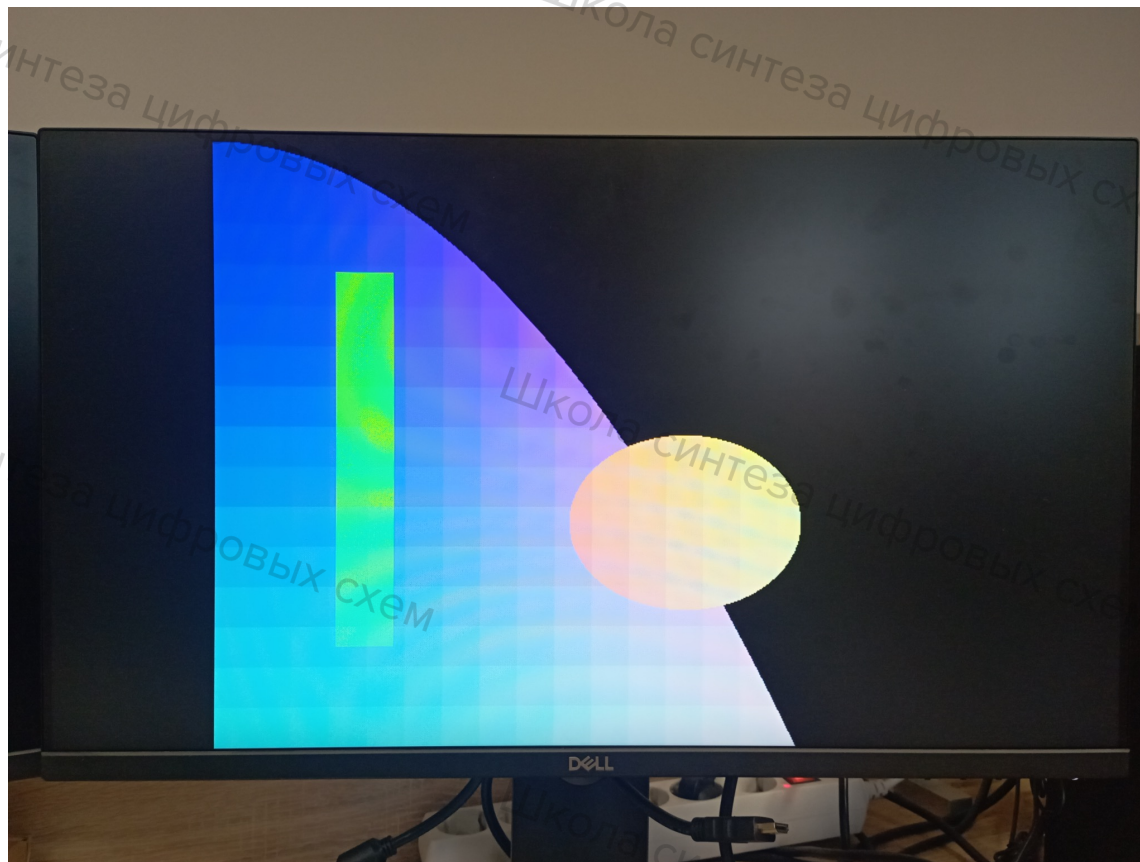
Имя подключаемого модуля (and_3, inv)

Название примитива. Например, нам может понадобиться 3 копии модуля and_3. Тогда мы сможем подключить 3 экземпляра модуля **and_3**, используя различные наименования для прототипов (andgate_1, andgate_2 ...)

Символ точка, перед наименованием порта отсылает к реальному порту подключаемого модуля. **В скобках** обозначается, куда будут подключаться сигналы в top-модуле



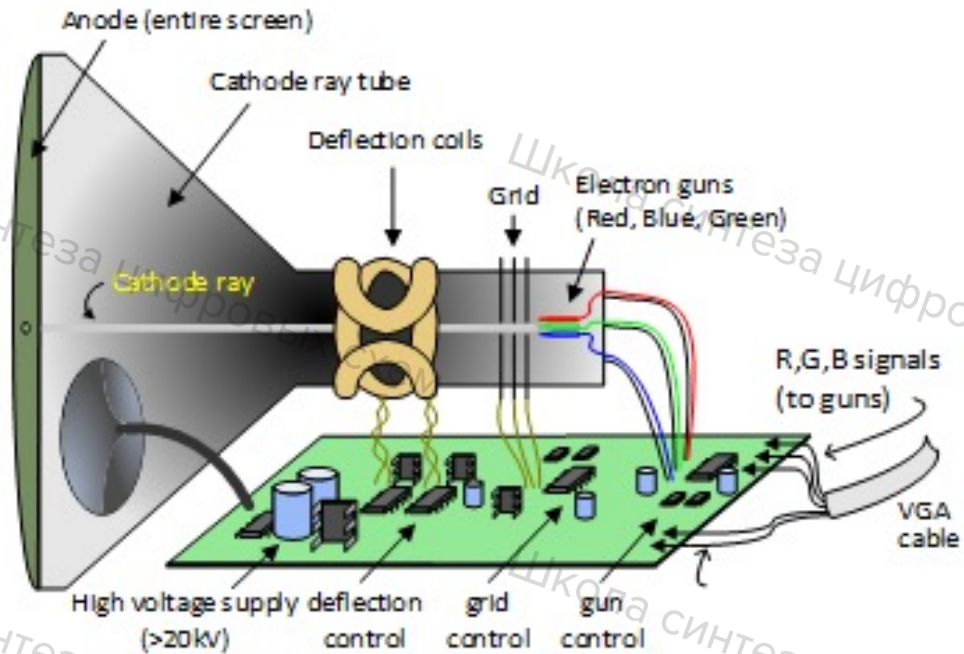
УПРАЖНЕНИЕ С VGA 2_1_rectangle_ellipse_parabola





УПРАЖНЕНИЕ С VGA 2_1_rectangle_ellipse_parabola

Digilent Nexys A7 Reference Manual

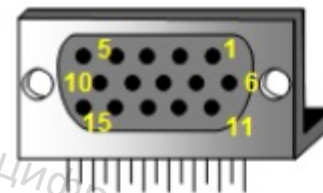


УПРАЖНЕНИЕ С VGA 2_1_rectangle_ellipse_parabola

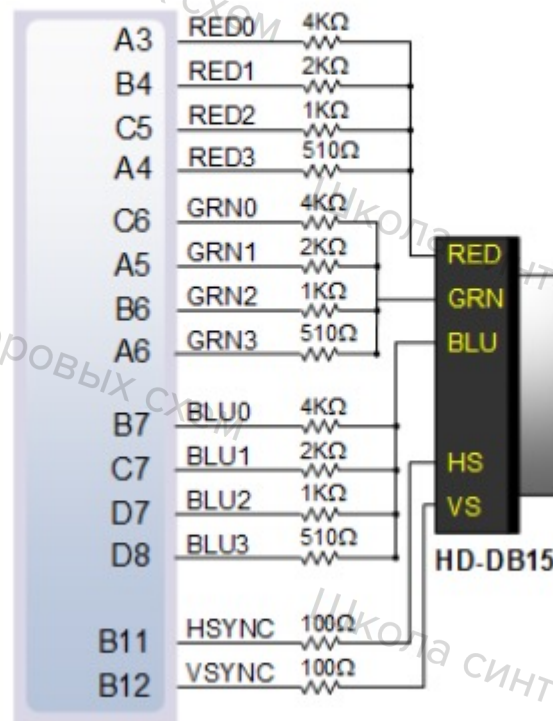
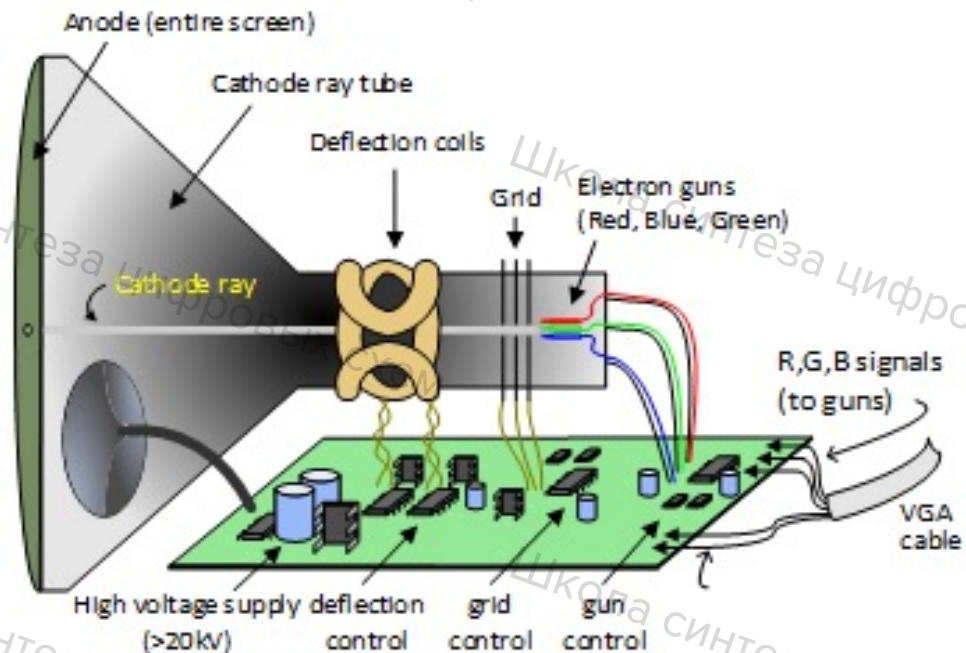
Digilent Nexys A7 Reference Manual



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ



Pin 1: Red	Pin 5: GND
Pin 2: Grn	Pin 6: Red GND
Pin 3: Blue	Pin 7: Grn GND
Pin 13: HS	Pin 8: Blu GND
Pin 14: VS	Pin 10: Sync GND

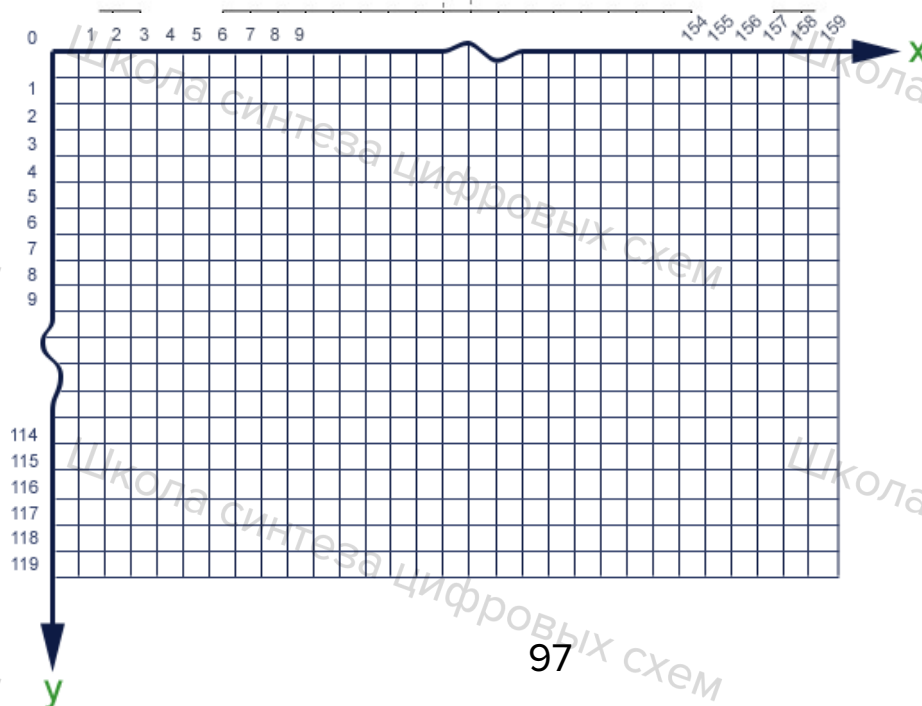
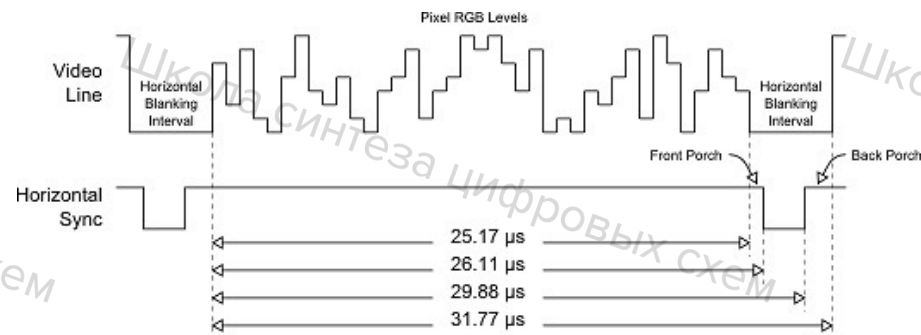
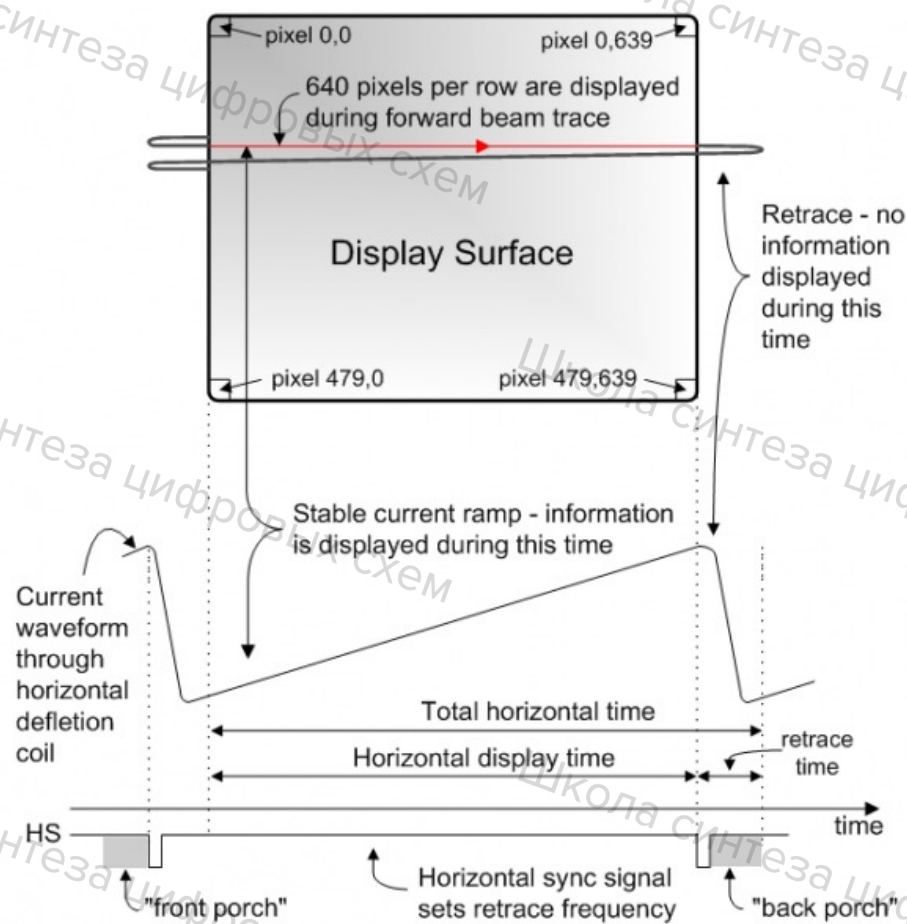


Artix-7



УПРАЖНЕНИЕ

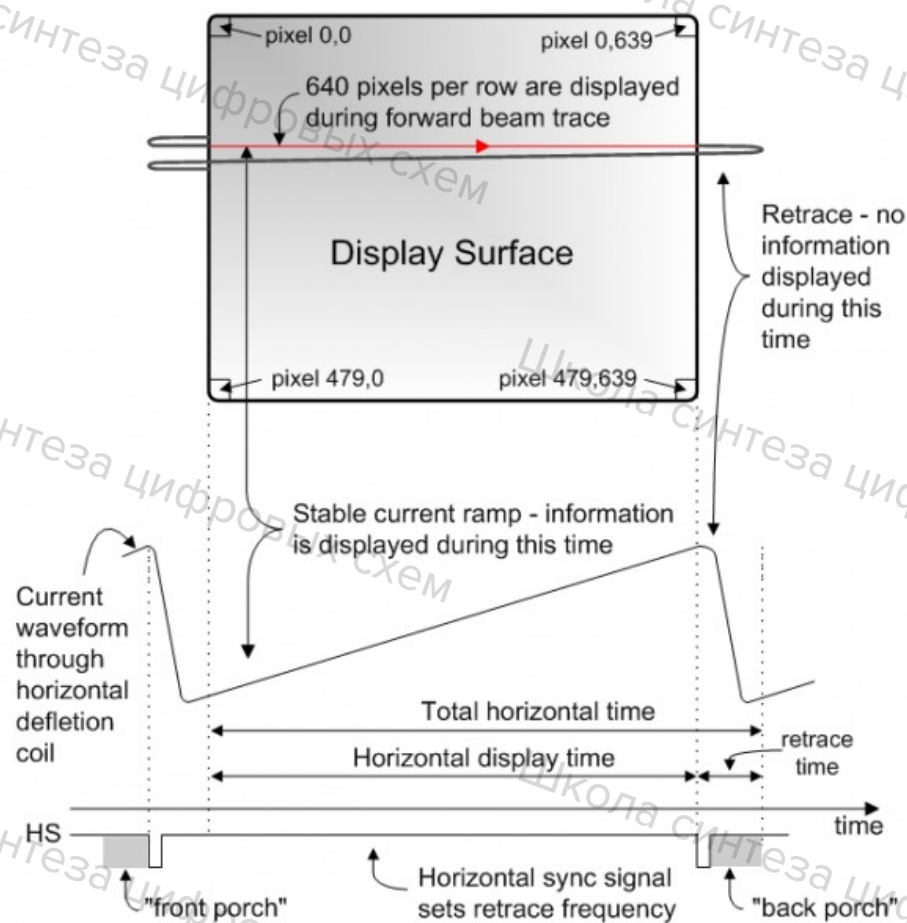
С VGA 2_1_rectangle_ellipse_parabola





УПРАЖНЕНИЕ

С VGA 2_1_rectangle_ellipse_parabola





УПРАЖНЕНИЕ

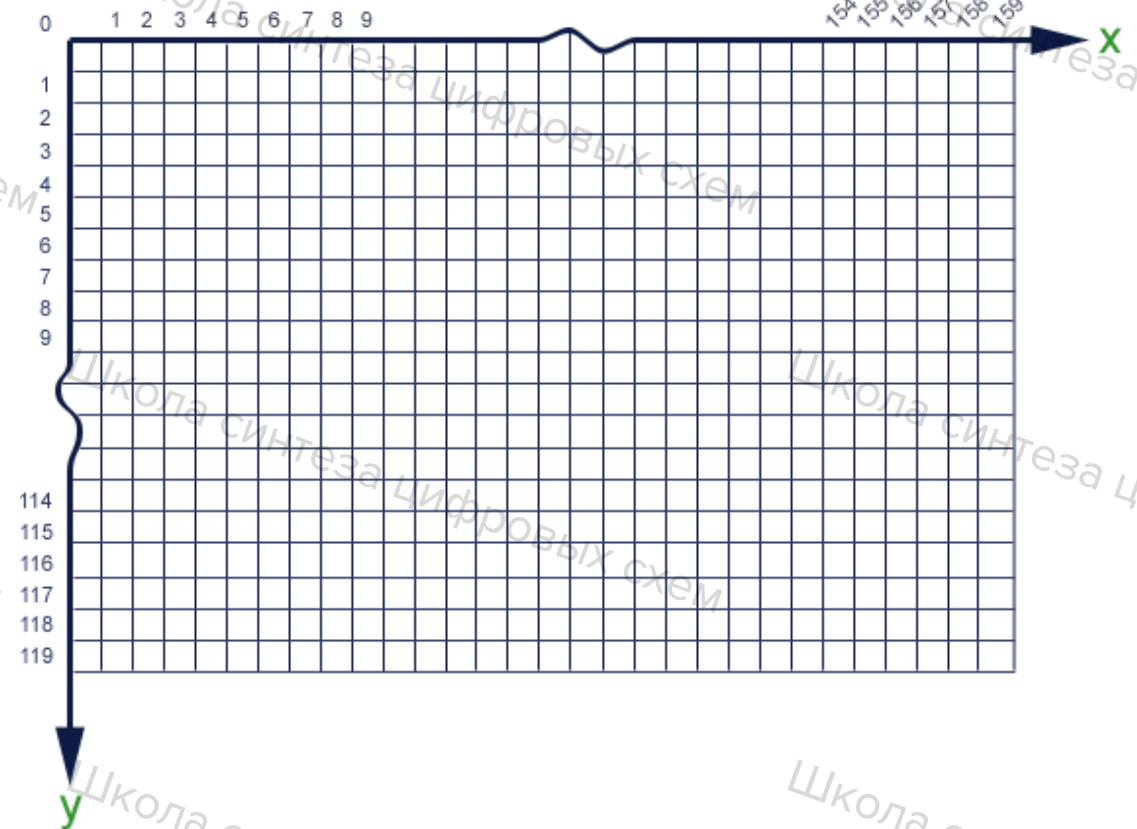
С VGA 2_1_rectangle_ellipse_parabola

```

57 //-----
58
59 wire display_on;
60
61 wire [w_x - 1:0] x;
62 wire [w_y - 1:0] y;
63
64 vga
65 # (
66     .HPOS_WIDTH ( w_x      ),
67     .VPOS_WIDTH ( w_y      ),
68
69     .CLK_MHZ     ( clk_mhz )
70 )
71 i_vga
72 (
73     .clk      ( clk      ),
74     .rst      ( rst      ),
75     .hsync    ( hsync    ),
76     .vsync    ( vsync    ),
77     .display_on ( display_on ),
78     .hpos     ( x        ),
79     .vpos     ( y        )
80 );
81

```

Счетчики координат





УПРАЖНЕНИЕ

С VGA 2_1_rectangle_ellipse_parabola

```

57 //-----
58
59 wire display_on;
60
61 wire [w_x - 1:0] x;
62 wire [w_y - 1:0] y;
63
64 vga
65 # (
66     .HPOS_WIDTH ( w_x ),
67     .VPOS_WIDTH ( w_y ),
68
69     .CLK_MHZ ( clk_mhz )
70 )
71 i_vga
72 (
73     .clk ( clk ),
74     .rst ( rst ),
75     .hsync ( hsync ),
76     .vsync ( vsync ),
77     .display_on ( display_on ),
78     .hpos ( x ),
79     .vpos ( y )
80 );
81

```

Описание генерации
фигур на экране.

Для каждого пикселя
с координатами X и Y
определяется его цвет

```

always_comb
begin
    red   = '0;
    green = '0;
    blue  = '0;

    if ( x >= screen_width / 2
        & x < screen_width * 2 / 3
        & y >= screen_height / 2
        & y < screen_height * 2 / 3 )
    begin
        if (key [0])
            green = '1;
        else
            green = x11[$left(x11) - 1 -: w_green];
    end

    `ifdef YOSYS
    if (x * x + 2 * y * y < screen_width * screen_width / 4) // Ellipse
    `else
    if (x ** 2 + 2 * y ** 2 < (screen_width / 2) ** 2) // Ellipse
    `endif
    begin
        red = x11[$left(x11) - 1 -: w_red];
    end

    if (x_2[w_x +: w_y] < y) // Parabola
        blue = key [1] ? '1 : y10[$left(y10) -: w_blue];
    end
end

```



УПРАЖНЕНИЕ

С VGA 2_1_rectangle_ellipse_parabola

```
logic [31:0] a_vect;  
logic [0:31] b_vect;  
logic [63:0] dword;  
integer sel;
```

```
a_vect[0 +:8] // ==a_vect[7:0]  
a_vect[15 -:8] // ==a_vect[15:8]  
  
b_vect[0 +:8] // ==b_vect[0:7]  
b_vect[15 -:8] // ==b_vect[8:15]  
  
dword[8*sel +:8] // variable part-select with fixed width
```

$w[x+:y]$ эквивалентно $w[x:(x+y-1)]$
 $w[x-y:]$ эквивалентно $w[x:(x-y+1)]$

`$left(cnt)` возвращает номер старшего Индекса в cnt. Если cnt 8-разрядный, То вернется 7.
Если `$left(cnt) = 7`, а `w_led = 3`, то `[$left(cnt) -: w_led]` эквивалентно `[7:5]`



УПРАЖНЕНИЕ

С VGA 2_1_rectangle_ellipse_parabola

```
1  `include "config.svh"
2
3  module lab_top
4  # (
5      parameter clk_mhz      = 50,
6                      w_key   = 4,
7                      w_sw    = 8,
8                      w_led    = 8,
9                      w_digit  = 8,
10                     w_gpio   = 100,
11
12                     screen_width = 640,
13                     screen_height = 480,
14
15                     w_red      = 4,
16                     w_green    = 4,
17                     w_blue     = 4,
18
19                     w_x        = $clog2 ( screen_width ),
20                     w_y        = $clog2 ( screen_height ),
21 )
```

Параметры это константы которые могут быть переопределены при синтезе дизайн и позволяют создавать конфигурированный дизайн

screen_width = 640

screen_height = 480

Параметры определяющие размер генерируемого изображения и соответственно его размер по осям x и y



УПРАЖНЕНИЕ

С VGA 2_1_rectangle_ellipse_parabola

```

always_comb
begin
    red   = '0;
    green = '0;
    blue  = '0;

    if ( x >= screen_width / 2
        & x < screen_width * 2 / 3
        & y >= screen_height / 2
        & y < screen_height * 2 / 3 )
    begin
        if (key [0])
            green = '1;
        else
            green = x11 [$left (x11) - 1 -: w_green];
        end
    end

    `ifdef VGSYS
    if (x * x + 2 * y * y < screen_width * screen_width / 4) // Ellipse
    `else
    if (x ** 2 + 2 * y ** 2 < (screen_width / 2) ** 2) // Ellipse
    `endif
    begin
        red = x11 [$left (x11) - 1 -: w_red];
    end

    if (x_2 [w_x +: w_y] < y) // Parabola
        blue = key [1] ? '1 : y10 [$left (y10) -: w_blue];
    end
end

```

Тут например параметры **screen_width** и **screen_height** используются в определении границ генерируемых фигур

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ



- Для проверки ваших знаний получите индивидуальное задание от куратора кластера.
- Задание посвящено выводу изображения по интерфейсу VGA и основано на упражнении `2_1_rectangle_ellipse_parabola`
- Цель вывести картинку по заданию куратора на монитор по VGA модифицировав код примера.
- Покажите свое решение куратору и получите отметку о успешном выполнении задания.



Автор (разработчик материала) лекции — Силантьев Александр Михайлович

- Использование материалов и записи лекции и/или их частей без предварительного согласия не допускается.
- По вопросам использования материалов и записи лекции в коммерческих целях необходимо направить обращение в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com.
- По вопросам некоммерческого использования материалов и записи лекции обращение может быть направлено в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com, либо на адрес электронной почты автора Силантьева А.М. silantiev@org.miet.ru.
Такое обращение обязательно должно содержать описание цели использования.



ШКОЛА СИНТЕЗА
ЦИФРОВЫХ СХЕМ

Клик правой кнопкой —
изменить изображение